TechPost ELEVATED

DC – AI – Routing – Switching – Security – Wireless – Automation

Micro-Segmentation in DC with Apstra and Junos

An HPE Juniper Networking Techpost

Adam Jarvis

2024-08-13

Contents

1 Micro-Segmentation in DC with Apstra and Junos 4
1.1 Purpose of the Document . . . 4
1.2 Introduction . . . 4
1.2.1 Macro-Segmentation vs Micro-Segmentation . . . 4
1.2.2 What is Group Based Policy (GBP) in VXLAN Networks? . . . 5
1.2.3 Benefits And Considerations for VXLAN GBP for Data Centers . . . 6
1.2.4 Junos Devices that Support VXLAN Group-Based Policy (GBP) . . . 7
1.2.5 Ingress and Egress Enforcement in VXLAN GBP . . . 7
1.2.6 Backward compatibility . . . 8
1.2.7 GBP TAG assignment . . . 8
1.2.8 VXLAN GBP vs Apstra GBP Framework . . . 9
1.3 Apstra Concepts . . . 9
1.3.1 Configlets . . . 9
1.3.2 Property Sets . . . 10
1.3.3 System Tags . . . 10
1.4 How the VXLAN GBP Configlet Works . . . 10
1.5 Configlet Breakdown . . . 10
1.5.1 Setting GBP Profile . . . 10
1.5.2 Firewall Filter Creation . . . 11
1.5.3 Firewall Filter Application . . . 12
1.5.4 Entire Configlet . . . 12
1.6 Property Set Breakdown . . . 13
1.6.1 VXLAN GBP Property Set Schema . . . 13
1.7 How to use the Configlet in your Blueprint . . . 14
1.7.1 Overview of the deployment process . . . 14
1.7.2 Deployment Process . . . 14
1.7.3 Creating Your Property Set . . . 18
1.7.4 Importing Configlet to Blueprint . . . 22
1.7.5 Tagging Ports in Apstra to Enforce GBP . . . 24
1.7.6 Using System Node Tags . . . 24
1.8 End Result . . . 27
1.8.1 Commit-check Procedures . . . 27
1.9 Successful Deployment – End State . . . 29
1.9.1 Firewall Filter Configuration . . . 29
1.9.2 SGT to Interface Binding . . . 30
1.9.3 VXLAN-GBP Profile Activation . . . 30
1.10 Adding Additional GBP Filters in the Future . . . 30
1.11 Removing GBP Policies . . . 31
1.11.1 Removing GBP policy from an Interface . . . 31
1.11.2 Removing Specific GBP Firewall Filter Configuration . . . 32
1.12 Validate . . . 32
1.12.1 Command: show ethernet-switching table . . . 33
1.12.2 Command: show firewall . . . 33

1.13 Useful links 34
1.14 Glossary 34
1.15 Acknowledgments 35

List of Figures

1 Macro Segmentation Vs Micro Segmentation . . . 5
2 VXLAN-GBP uses reserved fields in the VXLAN header for Scalable Group Tags (SGTs). . . . 6
3 download the .json file . . . 15
4 click Create Configlet . . . 16
5 Select Import Configlet . . . 16
6 Select the downloaded configlet file . . . 17
7 afterwards simply click create . . . 17
8 The imported configlet will appear in the list of configlets . . . 18
9 Create Property Set . . . 18
10 create your new data structure . . . 19
11 Once created your property set will be available to be used. . . . 19
12 Import Property Set . . . 20
13 Select the property set you created and click Import . . . 21
14 You will now see this imported . . . 22
15 Import Configlet . . . 22
16 Select the configlet you imported to Apstra earlier . . . 23
17 You will now see the configlet imported. . . . 24
18 Update Node Tags . . . 25
19 Update Node Tags . . . 25
20 add the tags you like . . . 26
21 You will now see the tag assigned to the generic system . . . 26
22 you will see the new GBP statements for that interface. . . . 27
23 the Apstra commit check feature . . . 28
24 the Apstra commit check feature . . . 29
25 Select the node and click on . . . 31
26 Remove the GBP tag from the interface. . . . 32
27 Shows other traffic, not denied by GBP, continuing as normal . . . 33
28 show ethernet-switching table . . . 33
29 show firewall . . . 34

1 Micro-Segmentation in DC with Apstra and Junos

Adam Jarvis - 08/13/2024

Discover how to implement micro-segmentation in your data center using Juniper Apstra and VXLAN Group-Based Policy. This comprehensive guide walks you through the process of deploying fine-grained security controls at the fabric level, offering a powerful solution for modern network security challenges.

Article written by Adam Jarvis and Elisabeth Rodrigues.

1.1 Purpose of the Document

This document provides a comprehensive guide on implementing micro-segmentation in data center environments using Juniper Apstra. It outlines a step-by-step process for deploying micro-segmentation through configlets, tags, and custom telemetry within Apstra. The guide is designed to be applicable to any customer environment that utilizes hardware supporting the necessary features for micro-segmentation.

> Note: If you are only interested in how to implement the VXLAN GBP configlet jump to the section "How to use the Configlet in your Blueprint".

1.2 Introduction

Group-Based Policy (GBP) is a network policy framework that manages security and traffic based on groups of endpoints rather than individual devices. This method enhances flexibility and scalability in network management. By categorizing endpoints into groups, GBP allows for role-based access control, making it easier to implement consistent security policies across various devices and users, particularly in data centers.

1.2.1 Macro-Segmentation vs Micro-Segmentation

To fully grasp the concepts in this document, it's crucial to understand the difference between macro-segmentation and micro-segmentation in network security:

Macro-Segmentation: This approach divides a network into large, broad segments or zones. It simplifies management by applying security policies to large groups of devices or applications, typically based on their role or function within the organization. Macro segmentation is implemented at Layer 3 boundaries, such as VLAN interfaces.

Micro-Segmentation: In contrast, micro-segmentation creates smaller, more precise segments within the network. It allows for fine-grained security controls at the individual workload or application level. This approach significantly enhances security by isolating different segments to prevent lateral movement of threats. Micro-segmentation provides segmentation between different groups of hosts that exist within the same Layer 3 segment.

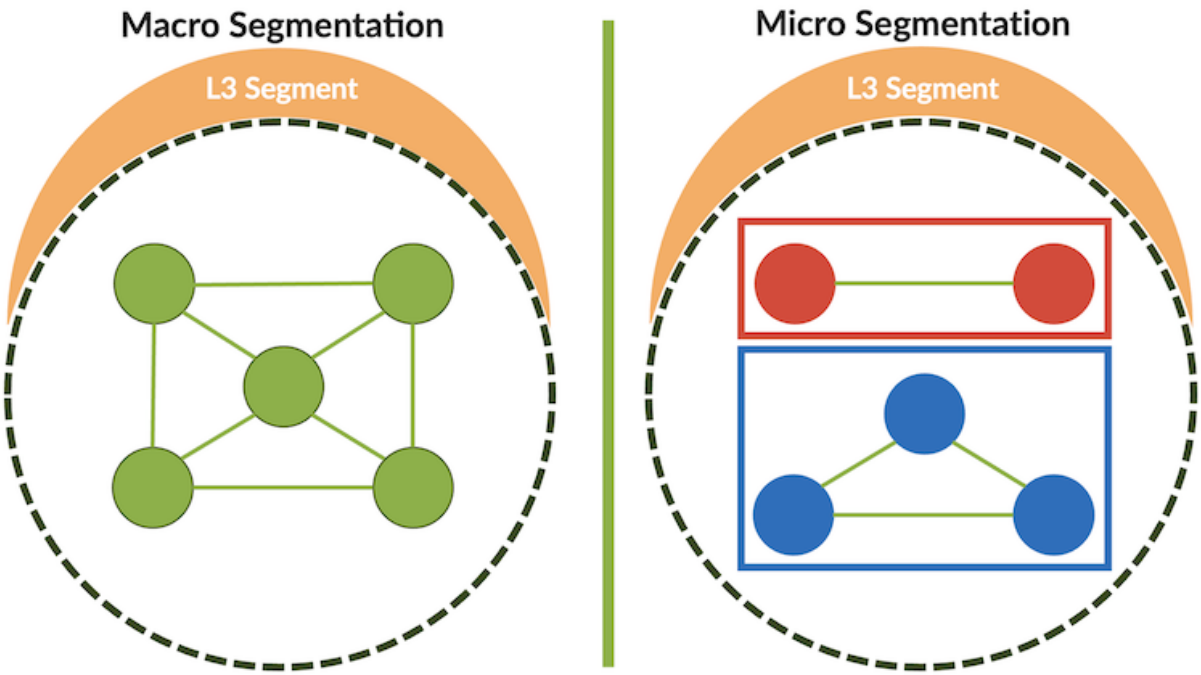


Figure 1: Macro Segmentation Vs Micro Segmentation

Macro-segmentation offers broad segmentation, simplifying policy application but providing less granular control. Micro-segmentation, on the other hand, enables detailed, workload-specific policies. While micro-segmentation enhances security by isolating segments to prevent internal threats, it requires more complex management. This document focuses on implementing micro-segmentation to secure the data center environment.

1.2.2 What is Group Based Policy (GBP) in VXLAN Networks?

Group-Based Policy (GBP) is an advanced network security framework that manages traffic and enforces security based on logical groups of endpoints rather than individual devices or IP addresses. This approach significantly enhances network flexibility and scalability, particularly when implemented in VXLAN (Virtual Extensible LAN) environments. GBP is described in this draft RFC "VXLAN Group Policy Option - draft-smith-vxlan-group-policy-00".

Key aspects of GBP in VXLAN networks include:

- Endpoint Grouping: Devices and users are categorized into logical groups based on their roles, functions, or security requirements, regardless of their physical location in the VXLAN fabric.
- Policy Definition: Security policies are defined between these groups, specifying allowed interactions, which are then enforced across the VXLAN overlay.
- Consistent Policy Application: Policies are consistently applied regardless of the physical location of devices within the network, a key benefit in VXLAN's location-independent architecture.
- Scalability: As new devices are added to a group, they automatically inherit the group's security policies, aligning well with VXLAN's ability to scale across large data center networks.

GBP facilitates micro-segmentation within VXLAN architectures by leveraging VXLAN's underlying technology. This integration provides location-independent endpoint access control, ensuring consistent security policies

throughout the network. As a result, network configuration is simplified by reducing the need for numerous firewall filters.

In VXLAN environments, GBP utilizes the VXLAN header to carry group policy information. Specifically, VXLAN-GBP uses reserved fields in the VXLAN header for Scalable Group Tags (SGTs).

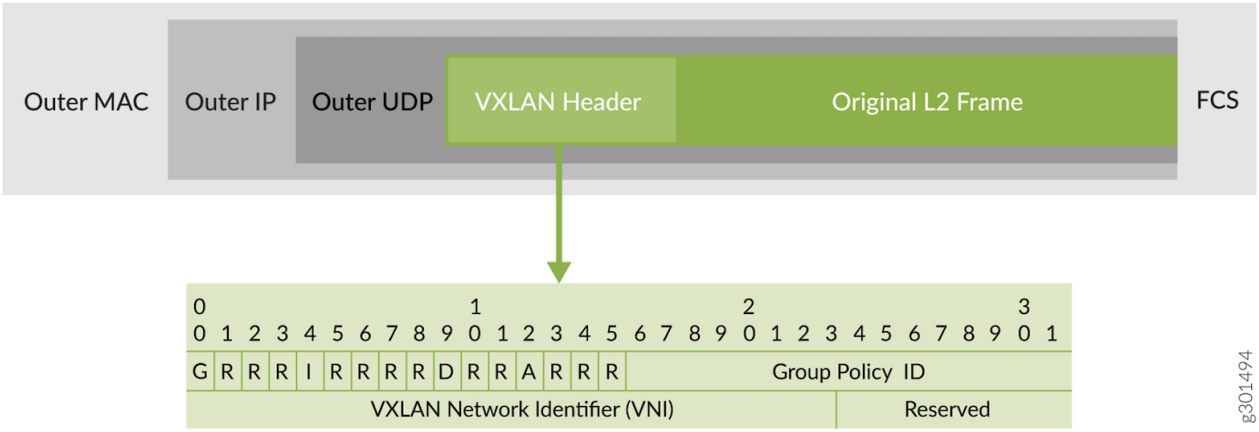


Figure 2: VXLAN-GBP uses reserved fields in the VXLAN header for Scalable Group Tags (SGTs).

These SGTs align with firewall filter rules, offering a robust alternative to traditional port or MAC address-based filtering. In data center environments, SGTs are typically assigned statically, providing consistent and predictable policy enforcement. This static assignment aligns well with the relatively stable nature of data center workloads and their security requirements. This allows for efficient policy enforcement at the network edge, reducing the need for complex, distributed firewall rules. The combination of GBP and VXLAN enables:

- Micro-segmentation: Fine-grained security policies can be applied within the same Layer 2 or Layer 3 segment.
- Mobility: Security policies follow workloads as they move within the VXLAN fabric.
- Simplified management: Policies are defined based on logical groups rather than network topology, reducing complexity in large-scale environments.
- Consistency: Static SGT assignment ensures that security policies remain stable and predictable, crucial for maintaining a secure data center environment.

1.2.3 Benefits And Considerations for VXLAN GBP for Data Centers

VXLAN GBP offers several advantages in data center environments, particularly for organizations dealing with complex network segmentation requirements. By enabling policy definition based on logical groups rather than traditional IP or MAC-based approaches, GBP simplifies the management of large, dynamic data center networks. This group-based approach enhances security through role-based access control and fine-grained segmentation, ensuring that only authorized entities can access specific network segments.

One of the key benefits of VXLAN GBP is its ability to provide location-agnostic policy enforcement. As policies are tied to groups rather than network locations, workloads can be moved within the data center without requiring policy reconfiguration, supporting more agile and flexible operations. This is particularly valuable in modern data centers with dynamic workload placement and frequent changes.

However, it's important to note that while VXLAN GBP offers these benefits, it may not always be the optimal solution for micro-segmentation in all data center scenarios. In many cases, implementing micro-segmentation at the hypervisor level is preferable due to its greater flexibility and granularity. Hypervisor-based solutions can often provide more detailed control and are typically easier to manage in highly virtualized environments. Hypervisor-based solutions can also create policy between virtual machines or Kubernetes pods, even if they reside on the same physical server – something VXLAN GBP can not.

Additionally, when considering VXLAN GBP for fabric-level micro-segmentation, it's crucial to be aware of potential scalability limitations. The number of policies that can be implemented is constrained by the TCAM (Ternary Content-Addressable Memory) size of the switches in the fabric. This hardware limitation can impact the scalability of the solution, especially in very large or complex environments.

Despite these considerations, there are valid use cases for VXLAN GBP in the data center fabric. For instance, in environments with a mix of virtualized and bare-metal workloads, or in scenarios where consistent policy enforcement across diverse infrastructure is required, VXLAN GBP can provide a unified approach to network segmentation and security.

Ultimately, the decision to implement VXLAN GBP should be based on a careful evaluation of specific organizational needs, existing infrastructure, scalability requirements, and overall security strategy. While it offers significant benefits in terms of simplified management and consistent policy enforcement, it's essential to weigh these advantages against potential limitations and alternative approaches to ensure the most effective solution for your data center environment.

1.2.4 Junos Devices that Support VXLAN Group-Based Policy (GBP)

The following table lists Juniper devices that support GBP and are applicable to data center use cases.

Switch Model	From Version
QFX5120-32C	22.4R1
QFX5120-48Y	22.4R1
EX4650 Series	22.4R1
EX4400 Series	22.4R1

> Note: Switch model support as of August 2024. Refer to the latest Junos documentation for up-to-date switch support. The up-to-date list can be found here or on Feature Explorer, here.

1.2.5 Ingress and Egress Enforcement in VXLAN GBP

In the context of data centers, ingress and egress enforcement are two different methods for applying GBP to traffic entering and leaving the network, respectively. These methods help control access, protect against threats, and maintain data integrity.

- **Ingress Enforcement**: Ingress enforcement applies policies as packets enter the network. This method ensures that only authorized traffic is allowed in, reducing the load on the internal network and

protecting against external threats early. The primary advantages of ingress enforcement include early threat detection and mitigation, which helps maintain the integrity and security of the internal network by blocking unauthorized or malicious traffic at the entry point. Additionally, filtering unwanted traffic at the ingress reduces internal traffic load, leading to better network performance and less congestion. However, scalability is another concern, GBP policies need to be shared between all switches and GBP tags need to be learned for all hosts on all switches that perform the ingress policy enforcement. In order to learn GBP tags of all hosts, those are advertised along with T2/T5 EVPN routes, this advertisement is not needed for egress filtering.

- **Egress Enforcement**: Egress enforcement, on the other hand, applies policies to outgoing traffic, ensuring that data leaving the network complies with security and access control policies. This method helps prevent data exfiltration and unauthorized communication, providing a crucial layer of protection for sensitive information. Egress enforcement is generally simpler to manage because it focuses on outgoing traffic, reducing the complexity associated with configuring and maintaining policies for inbound traffic. This simplification makes it easier to implement and manage, leading to enhanced data security by controlling outgoing traffic and preventing data leaks. However, the delayed detection of threats is a downside, as egress enforcement addresses potential threats only after they have entered the network. This reactive approach can increase the risk of internal compromise, allowing threats more time to cause damage. Moreover, since all traffic is allowed to enter the network initially, switches will face a higher load compared to ingress enforcement. For egress enforcement, the GBP tag of the source host **must** be set at the ingress and written in the VXLAN header. For egress enforcement, there is no need to advertise the GBP tag community in the T2/T5 EVPN routes and there is no need to learn all tags of all hosts, reducing drastically the scaling requirements of each switch.

For the purposes of this document, the focus will be on egress enforcement. This approach ensures that outgoing traffic adheres to security and access control policies, offering significant advantages in terms of simplified policy management, enhanced data security, and better network performance. Egress enforcement strikes a balance between security and performance, making it an ideal choice for ensuring data integrity and protecting against data breaches in dynamic, high-performance data center environments. For more detailed information on the implementation of micro-segmentation using GBP, refer to Juniper Networks documentation.

1.2.6 Backward compatibility

If certain devices in the fabric don't support GBP, they won't assign GBP tags or apply GBP filters, they will ignore any GBP information in the packets. It is entirely possible to have a mix of GBP and non-GBP devices within the same fabric.

1.2.7 GBP TAG assignment

GBP tag assignment can be done at several levels: MAC address, interface, vlan, IP prefix. By default, a MAC-based GBP filter only applies to switched traffic, and an IP-based GBP filter only applies to routed traffic. Starting in Junos OS Release 24.2R1, MAC-based GBP filters can also apply to routed traffic, and IP-based GBP filters can also apply to switched traffic. In this document, we will focus on interface-based GBP and we will leverage the Apstra tagging system to assign GBP tags to interfaces.

1.2.8 VXLAN GBP vs Apstra GBP Framework

If you are a previous user of Apstra, you may be familiar with the term Group Based Policy (GBP). However, it's important to note that this terminology has evolved. Apstra's feature, previously known as GBP, is now called Policy Assurance. This change in naming helps distinguish it from VXLAN GBP, which is a separate concept.

Apstra's Policy Assurance framework provides a fully automated solution for macro segmentation, integrating policy assurance directly into its platform. This approach ensures consistent policy implementation and simplifies operations through automated validation and continuous assurance. On the other hand, VXLAN GBP, which is the focus of this document, offers a different approach to network segmentation. It uses scalable group tags (SGTs) within the VXLAN header to apply detailed security policies dynamically, providing fine-grained control and flexibility in diverse network environments. While Apstra's Policy Assurance excels in automated macro segmentation, VXLAN GBP is ideal for precise and dynamic micro-segmentation needs in the fabric. It's crucial to understand that VXLAN GBP is not built into the Apstra UI and requires the use of configlets for implementation.

For more information on Apstra's Policy Assurance framework, refer to this blog post on the Juniper site.

This document focuses specifically on VXLAN GBP, its implementation, and use cases, and will not be covering Apstra's built-in Policy Assurance feature.

1.3 Apstra Concepts

Now we have covered micro-segmentation and VXLAN GBP basics, there are some Apstra concepts that are key to understand.

1.3.1 Configlets

A configlet in Apstra is a powerful tool that allows administrators to push additional custom configurations to network devices beyond the predefined Apstra reference design. Configlets are essential because they enable the deployment of bespoke features or settings that are not included in the standard design, ensuring flexibility and customization for specific network requirements.

Configlets can utilize Jinja templates, which enable dynamic and flexible configurations by pulling in data from device contexts and property sets. This allows the use of variables and loops to create adaptable configurations tailored to specific needs. Additionally, configlets can use tags to selectively apply configurations based on your own custom match conditions.

The primary advantage of using configlets is their ability to simplify complex configurations and automate the deployment process. They can be designed to accommodate various vendor styles, such as NX-OS, EOS, Junos, and SONiC, making them versatile across different network environments. Configlets can be applied at different points in the configuration process, ensuring they integrate seamlessly with the overall network setup.

In summary, Apstra configlets offer a legitimate and efficient way to implement additional configurations, leveraging Jinja templates, device contexts, property sets, and tags to provide flexible, scalable, and automated network management.

1.3.2 Property Sets

A property set in Apstra is a collection of data that defines specific properties, which can work in conjunction with configlets. Instead of embedding data directly into configlets which makes them challenging to maintain, property sets store variable values that can be referenced dynamically, providing flexibility for future changes. Written in JSON or YAML, property sets support various data structures, such as key-value pairs, lists, and dictionaries. They facilitate the customization of network configurations by allowing the separation of static and variable elements, making it easier to manage and update configurations.

1.3.3 System Tags

Tags in Apstra are user-defined metadata that can be applied to various network elements, such as systems, links, interfaces, and connectivity templates. They allow users to add additional information or labels to these elements, which then become part of the device context. Tags are particularly useful when using configlets, as they enable selective application of configurations based on the tagged attributes. This flexibility helps in managing and automating network configurations more efficiently by organizing and identifying network elements through meaningful labels.

1.4 How the VXLAN GBP Configlet Works

The previous section explained the theory of segmentation in the data center and some key Apstra concepts. This section will focus on the configlet, property set, and tags plus how to apply all the former to create your own customer VXLAN GBP deployment.

The configlet performs several tasks that collectively contribute to achieving micro-segmentation within your data center fabric. The three key points are: first, enabling VXLAN GBP; second, creating the firewall filters that define which tags can communicate with each other; and finally, adding the interface configuration to attach the GBP firewall filter policies to specific ports. This configlet leverages extensive Jinja coding, allowing it to be deployed once and left in place. Any changes only need to be made to property sets and system node tags, simplifying maintenance and updates.

1.5 Configlet Breakdown

Here we break down the configlet into smaller sections and add comments to provide some context on how it works

1.5.1 Setting GBP Profile

This Junos configuration sets up the switch to support group-based policies (GBP) for Layer 2/Layer 3 traffic in an EVPN-VXLAN environment.

Chassis Configuration: The chassis block allocates resources for the vxlan-gbp-profile, enabling the switch to handle VXLAN group-based policies. This is crucial for ensuring the hardware can process GBP-related tasks.

This configuration is necessary to effectively manage and enforce group-based policies within an EVPN-VXLAN environment, ensuring proper traffic handling and security compliance.

```
{# COMMENT_ONLY: Allocate switch resources for L2/L3 group-based policies #}
chassis {
   forwarding-options {
      vxlan-gbp-profile;
   }
}
```

> Note: Applying the vxlan-gbp-profile will cause the PFE to restart, which will mean the switch does not forward traffic for a short period. More details can be found here.

1.5.2 Firewall Filter Creation

This Junos configuration defines a firewall filter named MSEG for the family any. Within this filter, multiple terms are created dynamically based on the combination of source and destination Scalable Group Tags (SGTs). Each term specifies conditions using gbp-src-tag and gbp-dst-tag, and an associated action. The configuration also includes counters for each term to track traffic between specific source and destination tags. This approach allows for detailed and scalable security policy enforcement, ensuring precise control over network traffic based on group-based policies.

```
{# COMMENT_ONLY: Initializes a namespace object 'data' with an empty list attribute '
    src_tag_list' #}
{% set data = namespace(src_tag_list=[]) %}

{# COMMENT_ONLY: Creation of the firewall filters needed for GBP, defined by the property set
    imported #}

firewall {
   family any {
     filter MSEG {
{# COMMENT_ONLY: Property Set root is gbp_info #}
{% for all_filters in gbp_info %}
   {# COMMENT_ONLY: Looping through the property set #}
   {% for src_tag, src_tag_filters in all_filters.iteritems() %}
      {# COMMENT_ONLY: Adding the src tag to the data list for later steps #}
      {% if src_tag not in data.src_tag_list %}
         {% set data.src_tag_list = data.src_tag_list + [src_tag] %}
      {% endif %}
      {# COMMENT_ONLY: looping through the nested items, dst tag and action to craeate the
          filters #}
      {% for dst_tag_info in src_tag_filters %}
         {% for dst_tag, action in dst_tag_info.items() %}
      {# COMMENT_ONLY: Junos firewall filter config #}
      term From{{src_tag}}-To{{dst_tag}} {
         from {
            gbp-src-tag {{src_tag}};
            gbp-dst-tag {{dst_tag}};
         }
         then {
            {{action}};
            count {{src_tag}}-To{{dst_tag}};
            }
         }
         {% endfor %}
      {% endfor %}
   {% endfor %}
{% endfor %}
      }
   }
}
```

1.5.3 Firewall Filter Application

This Junos configuration dynamically attaches group-based policy (GBP) tags to specific interfaces. The configuration iterates through a list of unique source tags and applies firewall filter terms to each interface matching the tags. For each interface, if it belongs to an aggregated Ethernet (AE) bundle, the configuration sets the filter terms accordingly. If the interface is not part of an AE bundle, it applies the filter directly to the non-AE interface. This ensures that each relevant interface enforces the GBP policies effectively, enhancing network segmentation and security.

```
{# COMMENT_ONLY: Taking the data list and removing any duplicate values #}
{% set unique_src_tag_list= data.src_tag_list %}

{# COMMENT_ONLY: Looping through the unique tag list #}
{% for src_tag in unique_src_tag_list %}
   {# COMMENT_ONLY: Looping through each device interface (device context) #}
   {% for if_name,if_param in interface.items() %}
      {# COMMENT_ONLY: Searching is the link has a tag that has gbp_ and the src tag #}
      {% for tags in if_param['intf_tags'] %}
         {% if tags == "gbp_"+src_tag %}
            {# COMMENT_ONLY: Matches an AE interface and sets the AE interface config #}
            {% if if_param['part_of'] %}
set firewall family any filter GBP-TAG term TAG{{src_tag}}-{{if_param['part_of']}} from interface {{if_param['part_of']}}.0
set firewall family any filter GBP-TAG term TAG{{src_tag}}-{{if_param['part_of']}} then gbp-tag {{src_tag}}
            {% else %}
               {# COMMENT_ONLY: Matches an AE interface and sets the non-AE interface config #}
               {% if "ae" not in if_param['intfName'] %}
set firewall family any filter GBP-TAG term TAG{{src_tag}}-{{if_param['intfName']}} from interface {{if_param['intfName']}}.0
set firewall family any filter GBP-TAG term TAG{{src_tag}}-{{if_param['intfName']}} then gbp-tag {{src_tag}}
               {% endif %}
            {% endif %}
         {% endif %}
      {% endfor %}
   {% endfor %}
{% endfor %}
```

1.5.4 Entire Configlet

For the latest configlet use the Apstra configlet GitHub repository - https://github.com/Juniper-SE/Apstra-configlets/tree/main/Juniper/JUNOS/VXLAN_Interface_Based_GBP

```
chassis {
   forwarding-options {
      vxlan-gbp-profile;
   }
}

{% set data = namespace(src_tag_list=[]) %}

firewall {
   family any {
     filter MSEG {
{% for all_filters in gbp_info %}
   {% for src_tag, src_tag_filters in all_filters.iteritems() %}
      {% if src_tag not in data.src_tag_list %}
         {% set data.src_tag_list = data.src_tag_list + [src_tag] %}
      {% endif %}
      {% for dst_tag_info in src_tag_filters %}
         {% for dst_tag, action in dst_tag_info.items() %}
      term From{{src_tag}}-To{{dst_tag}} {
         from {
            gbp-src-tag {{src_tag}};
            gbp-dst-tag {{dst_tag}};
         }
         then {
            {{action}};
            count {{src_tag}}-To{{dst_tag}};
```

```
            }
         }
         {% endfor %}
      {% endfor %}
   {% endfor %}
{% endfor %}
      }
   }
}

{% set unique_src_tag_list= data.src_tag_list %}

firewall {
   family any {
      filter GBP-TAG {
      micro-segmentation;
{% for src_tag in unique_src_tag_list %}
   {% for if_name,if_param in interface.items() %}
      {% for tags in if_param['tags'] %}
         {% if tags == "gbp_"+src_tag %}
            {% if "ae" not in if_param['intfName'] and "ae" not in if_param['part_of'] %}
      term TAG{{src_tag}}-{{if_param['intfName']}} {
         from {
            interface {{if_param['intfName']}}.0;
         }
         then gbp-tag {{src_tag}};
      }
            {% elif "ae" in if_param['part_of']%}
      term TAG{{src_tag}}-{{if_param['part_of']}} {
         from {
            interface {{if_param['part_of']}}.0;
         }
         then gbp-tag {{src_tag}};
      }
            {% endif %}
         {% endif %}
      {% endfor %}
   {% endfor %}
{% endfor %}
      }
   }
}
```

> Note: Applying the vxlan-gbp-profile will cause the PFE to restart, which will mean the switch does not forward traffic for a short period. More details can be found here

1.6 Property Set Breakdown

The property set structure is fairly simple however the most important thing to note “gbp_info” is the root of the structure. The configlet is looking for “gbp_info” and without it you will receive an error.

1.6.1 VXLAN GBP Property Set Schema

```
gbp_info: <- the name gbp_info must be at the root of the YAML structure
- Src Tag: <- the source gbp tag (SGT Value)
  - Dst Tag: Action <- the dst gbp tag is the key (SGT Value), the action is the value
  - Dst Tag: Action
  - Dst Tag: Action
- Src Tag:
  - Dst Tag: Action
  - Dst Tag: Action
```

Example VXLAN GBP Property Set Structure:

```
gbp_info:
- '10':
  - '10': accept
  - '20': accept
  - '30': discard
- '20':
```

```
    - '20': accept
    - '10': accept
    - '30': discard
  - '30':
    - '10': discard
    - '20': discard
    - '30': accept
```

1.7 How to use the Configlet in your Blueprint

The previous section described the theory of the configlet and how to create a property set that is applicable to your own use case. This section will now focus on the deployment of this configlet/property set combination in your Apstra environment.

1.7.1 Overview of the deployment process

1. GitHub configlet download
2. Importing the configlet into your Apstra instance
3. Creating a property set in Apstra
4. Importing both the configlet and property set into your blueprint
5. Assigning node tags to apply the GBP interface configuration.

1.7.2 Deployment Process

GitHub Repo:

The first step in using the group-based policy configlet is to download the latest version from the juniper GitHub repo. This is important as it's constantly updated and this document may eventually become slightly out of date. The GitHub repository can be found here: https://github.com/Juniper-SE/Apstra-configlets/tree/main/Juniper/JUNOS/VXLAN_Interface_Based_GBP

Importing the Configlet to Apstra:

From the GitHub repository, you want to download the .json file as this is the complete configlet that can be imported directly into Apstra.

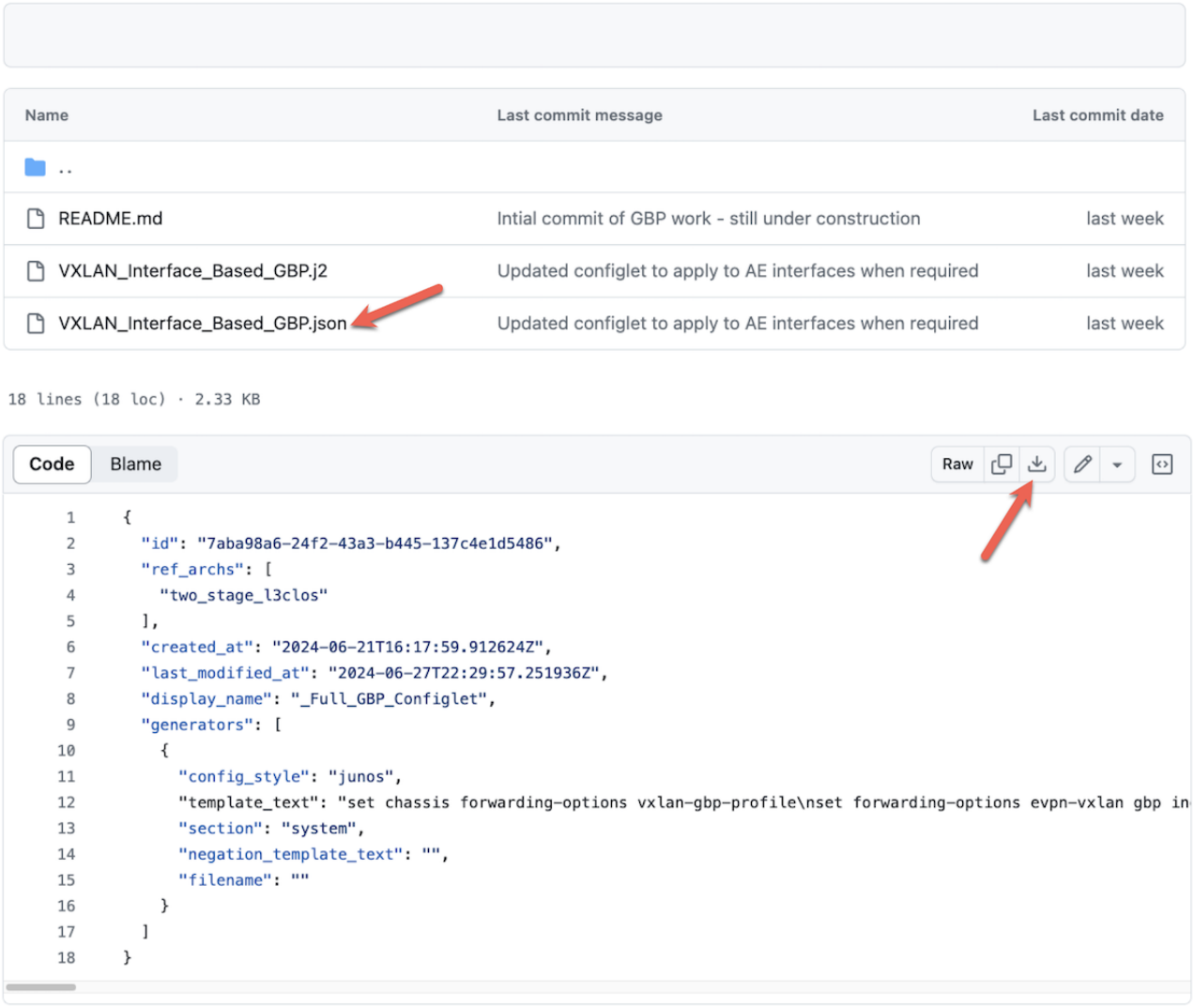


Figure 3: download the .json file

Once you have downloaded the configlet from the GitHub repository, it can now be imported directly into Apstra using the steps below.

The import is done under the **Design » Configlets** window.

1. Go to the configlet window under the design section and click **Create Configlet**, this is where you will do the import.

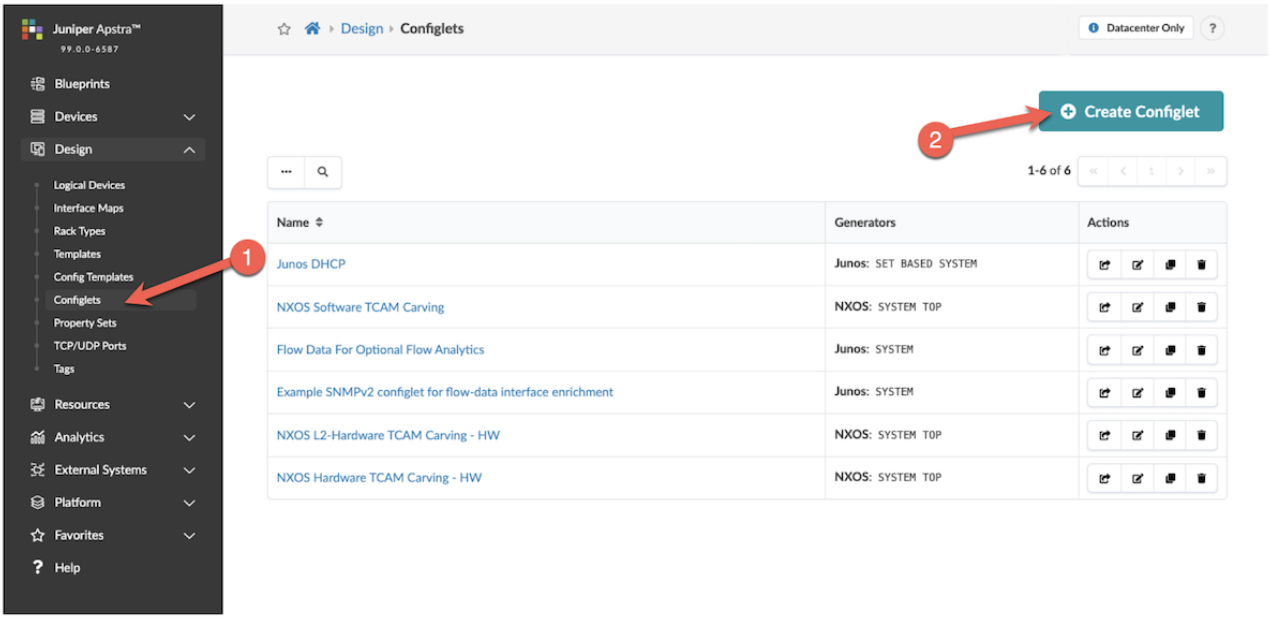


Figure 4: click Create Configlet

2. Select **Import Configlet**.

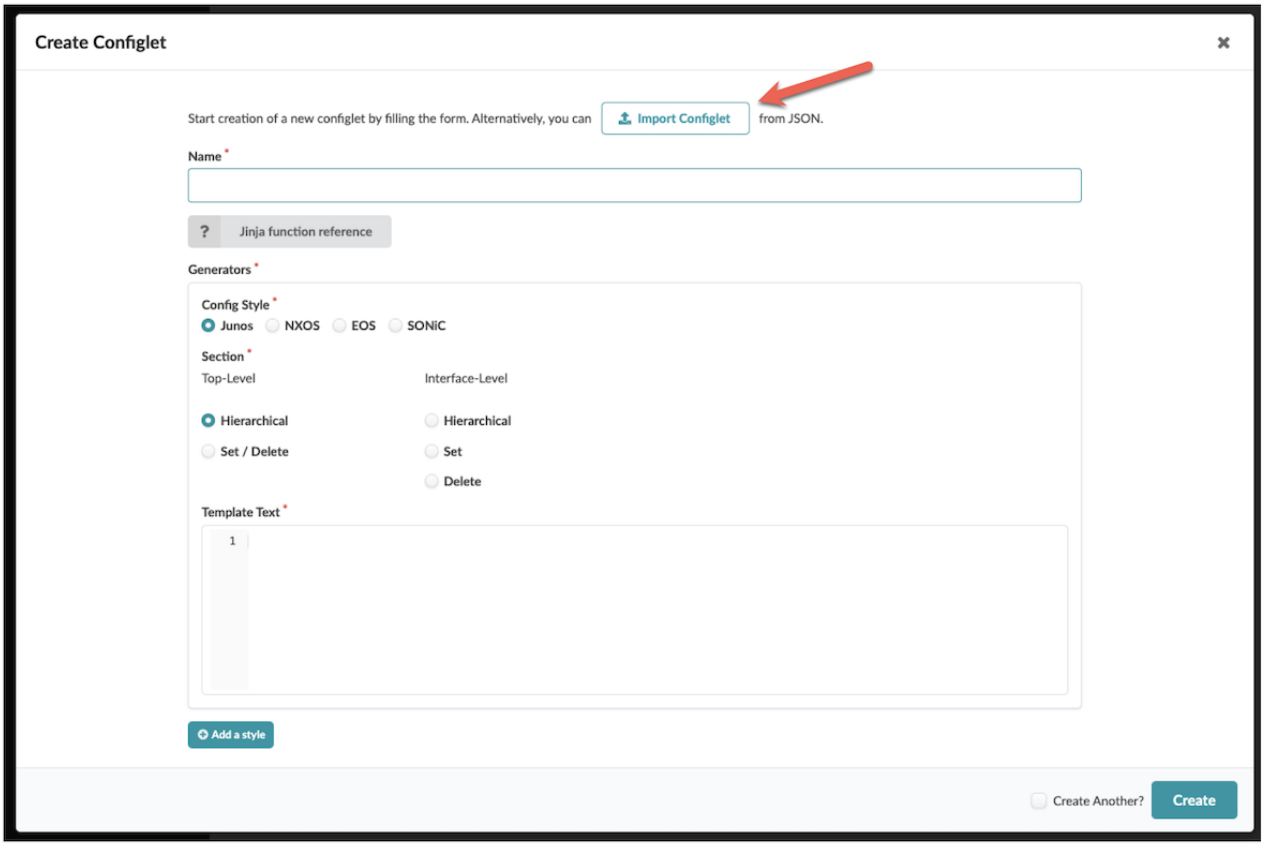


Figure 5: Select Import Configlet

3. Select the downloaded configlet file, this will be a .json file. While the configlet is a Jinja2 (.j2) file the import contains more than just the config itself.

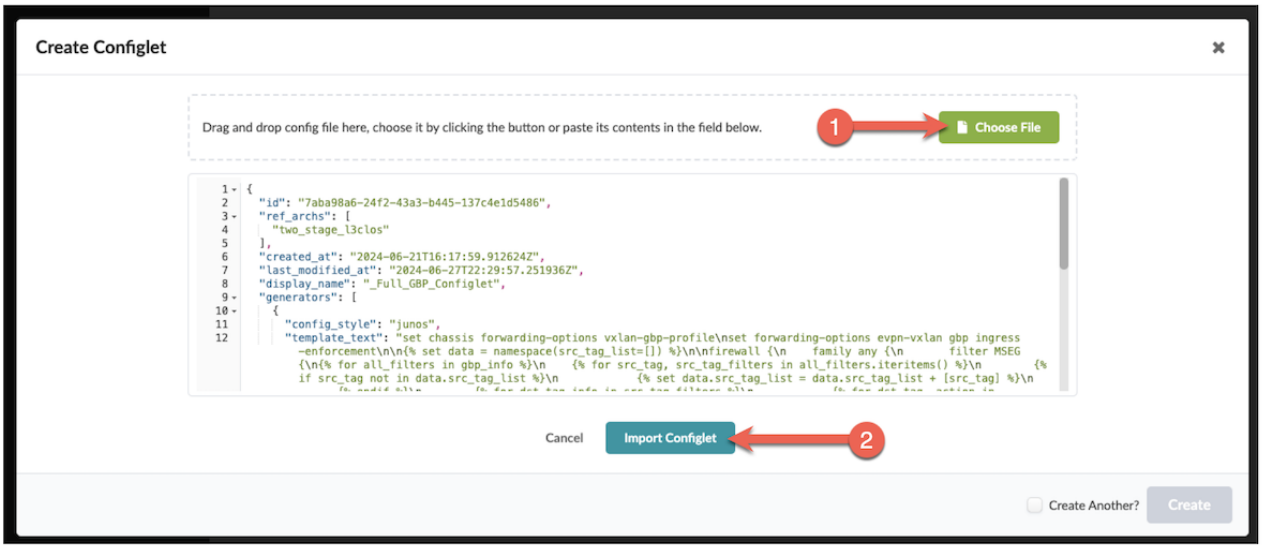


Figure 6: Select the downloaded configlet file

4. You will likely want to edit the name of the configlet but afterward simply click create.

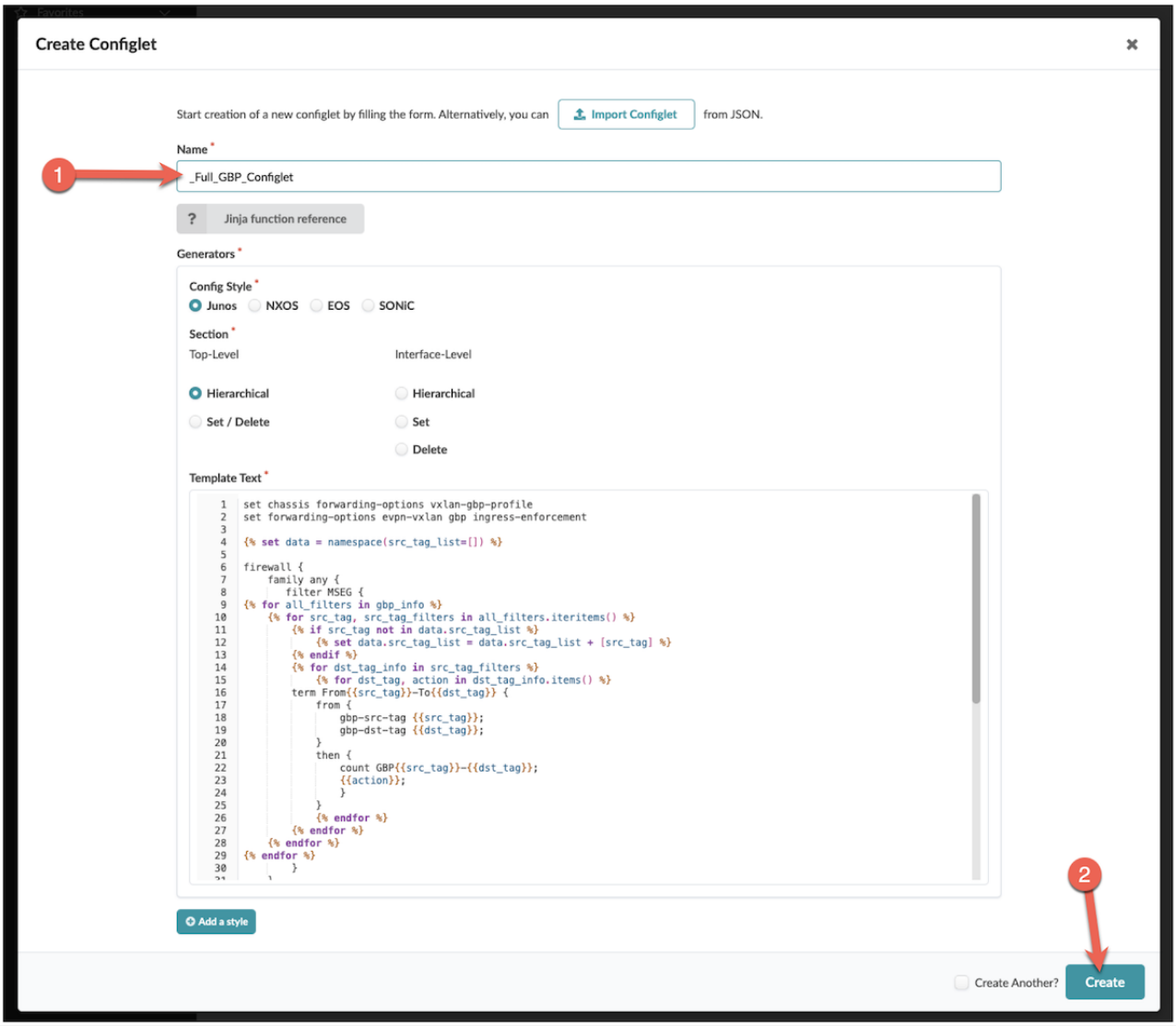


Figure 7: afterwards simply click create

5. The imported configlet will appear in the list of configlets

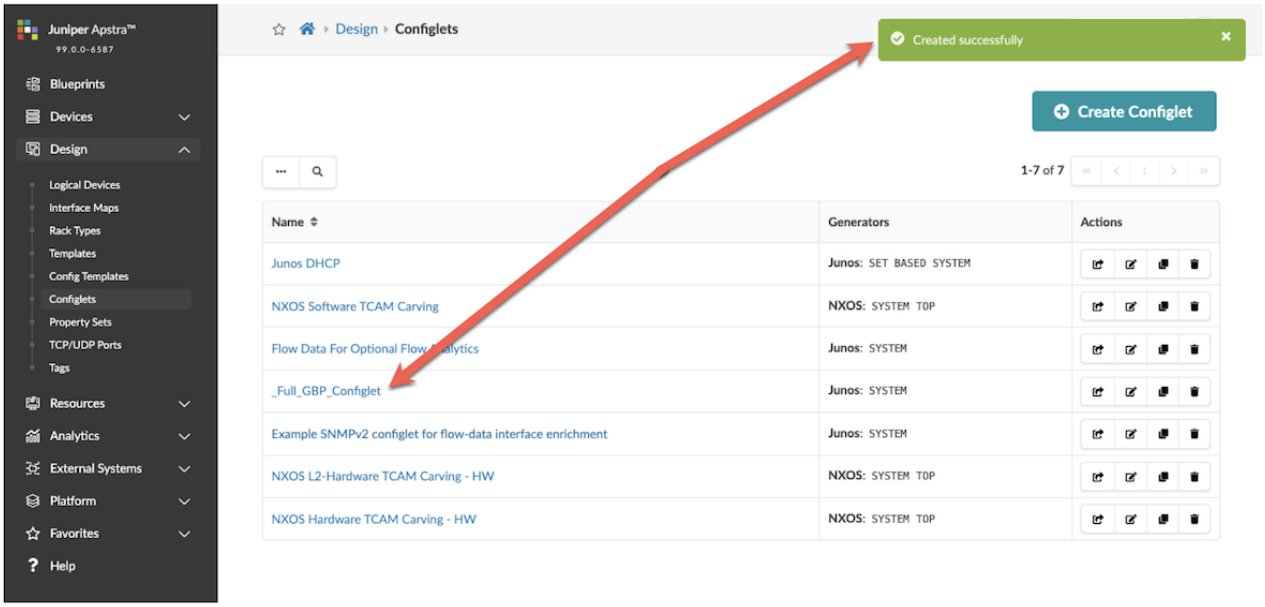


Figure 8: The imported configlet will appear in the list of configlets

1.7.3 Creating Your Property Set

Alongside the configlet imported above, a property set is required that contains all of the GBP information for your environment. The GitHub repository contains examples examples.

1. Go to the property set window under the design section and click **Create Property Set**, this is where you will do the import.

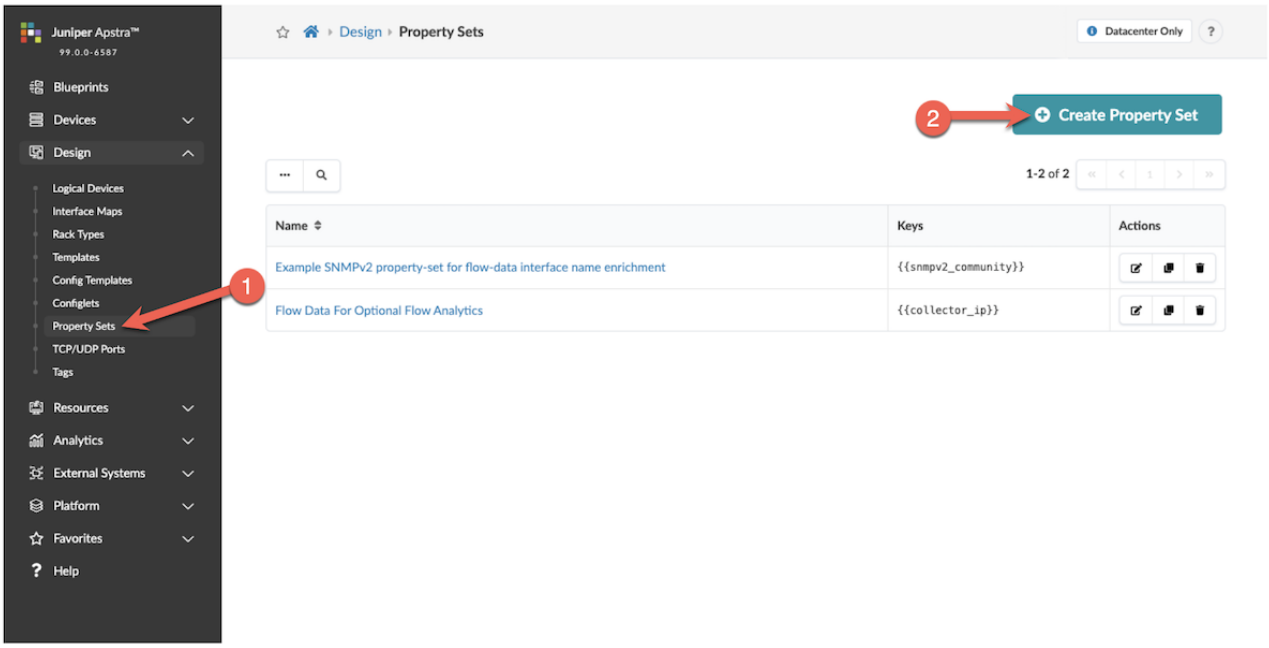


Figure 9: Create Property Set

2. Enter a name, select if this will be in the JSON or YAML format, create your new data structure, and finally click **Create**. The example used in this paper is a YAML structure.

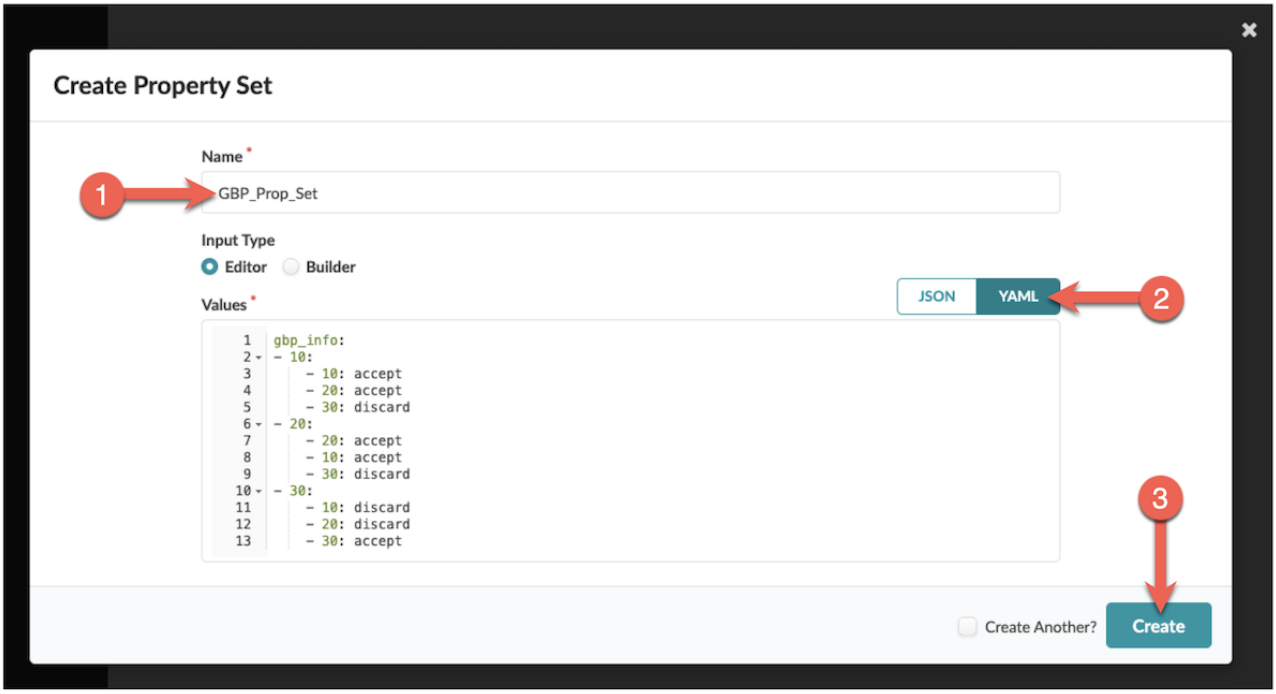


Figure 10: create your new data structure

3. Once created your property set will be available to be used.

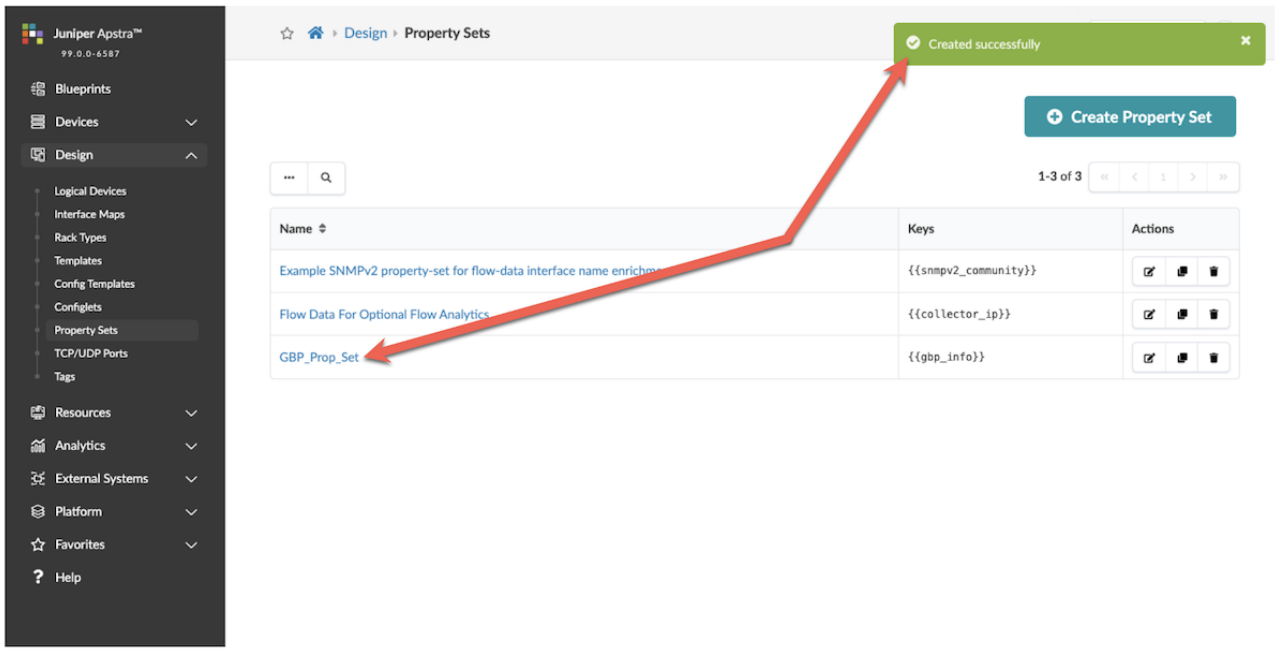


Figure 11: Once created your property set will be available to be used.

Importing Property Set to Blueprint

Once both the configlet and property set have been imported/created, they can now be added to your blueprint. It does not matter in which order you add them to the blueprint, but you will need to add both

otherwise you'll receive an error.

1. Go to **Staged » Catalog » Property Sets** within the blueprint and select **Import Property Set**.

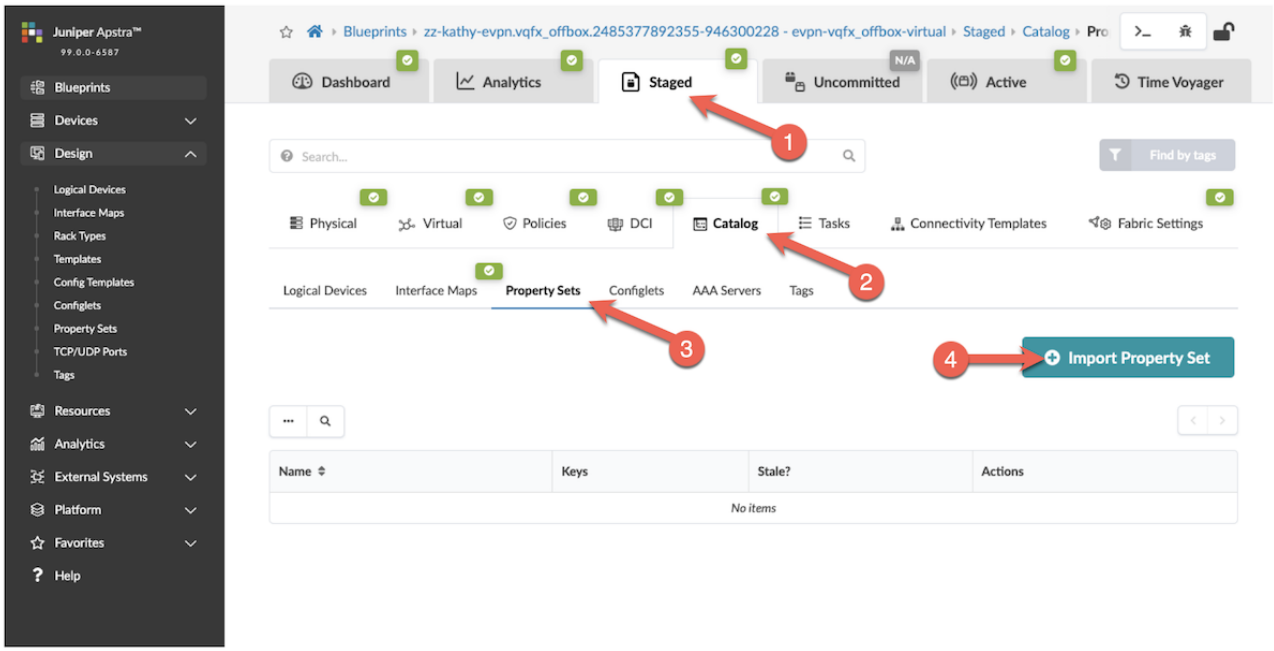


Figure 12: Import Property Set

2. Select the property set you created and click **Import**.



Figure 13: Select the property set you created and click Import

3. You will now see this imported.

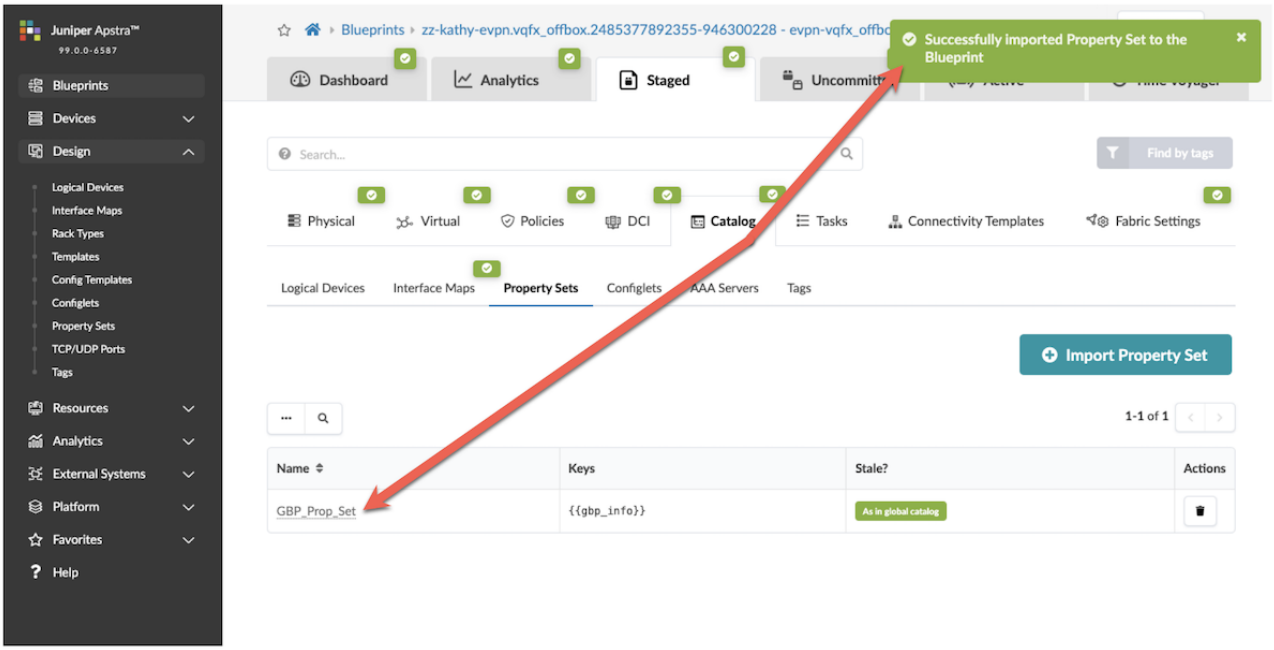


Figure 14: You will now see this imported

1.7.4 Importing Configlet to Blueprint

1. Go to **Staged » Catalog » Configlets** within the blueprint and select **Import Configlet**.

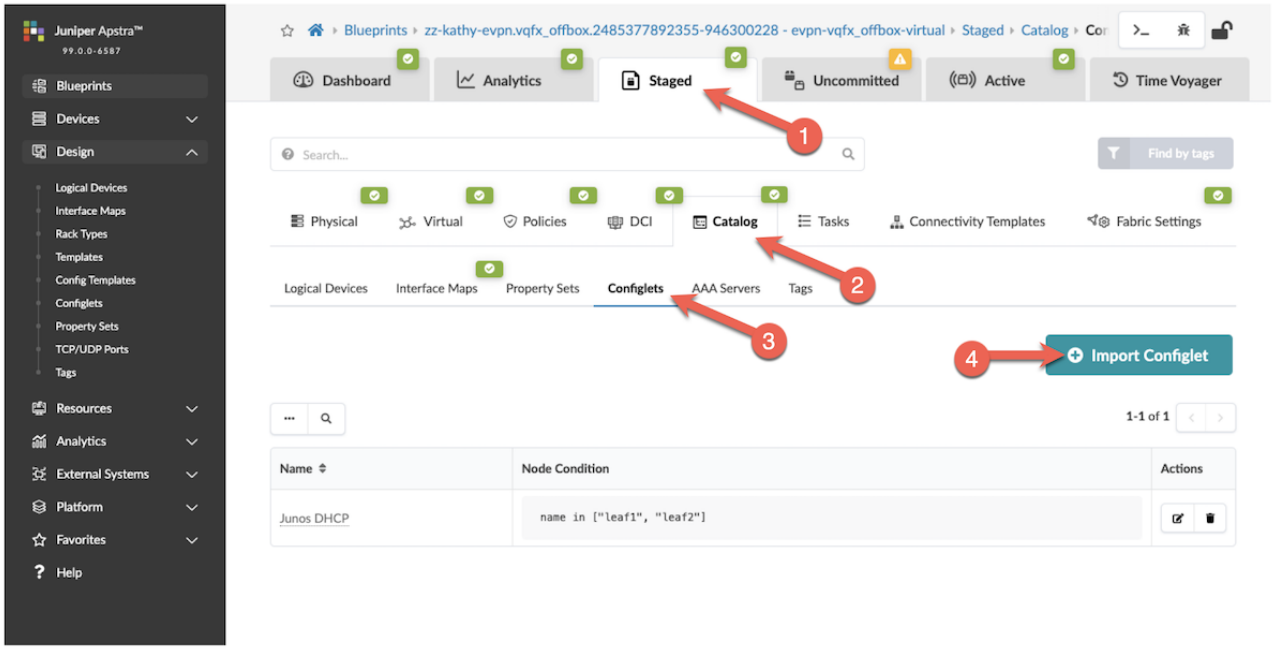


Figure 15: Import Configlet

2. Select the configlet you imported to Apstra earlier, select where to apply this configlet, and click **Import**. In this example, I am choosing to apply this to every leaf switch in the blueprint.

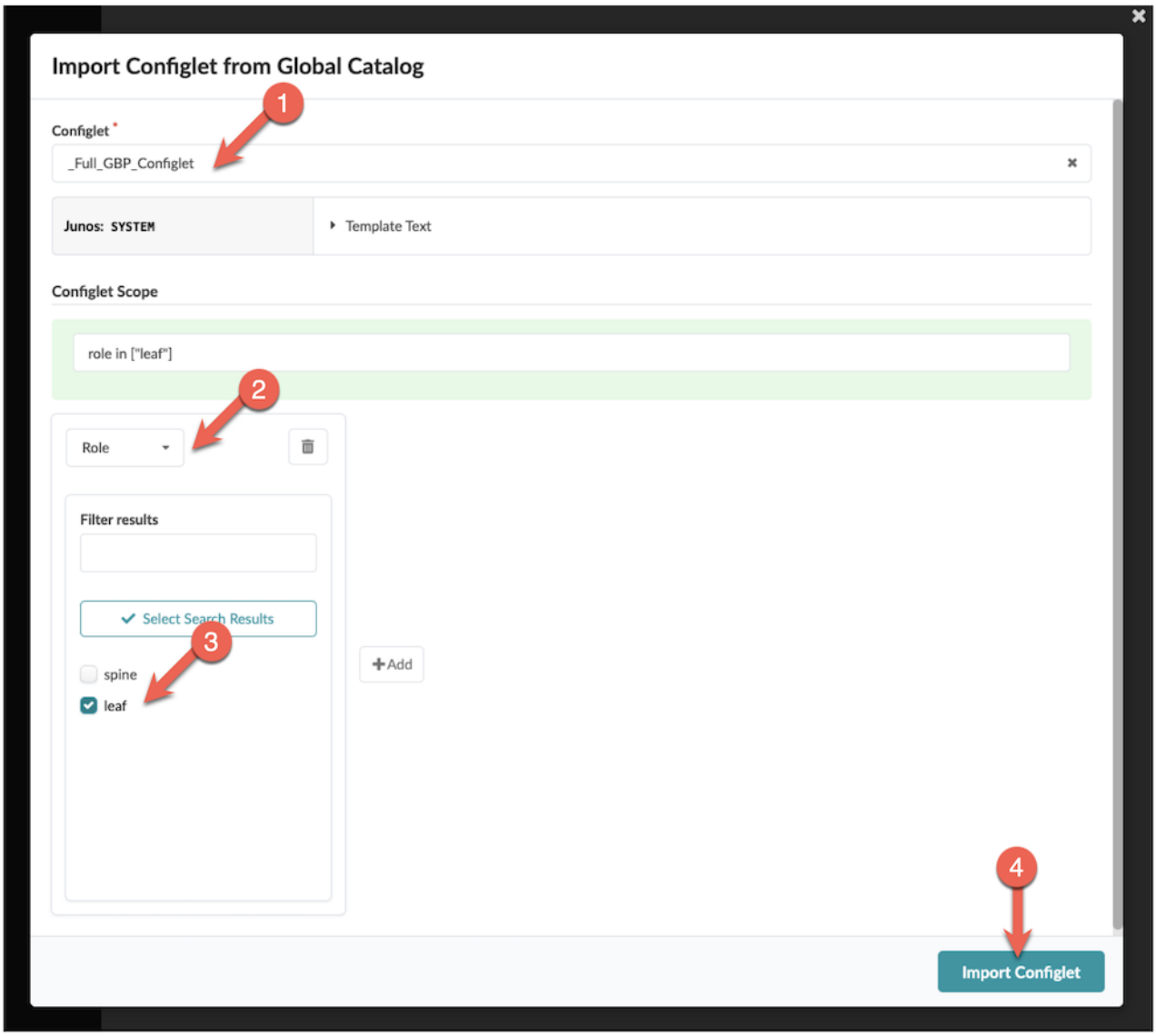


Figure 16: Select the configlet you imported to Apstra earlier

3. You will now see the configlet imported.

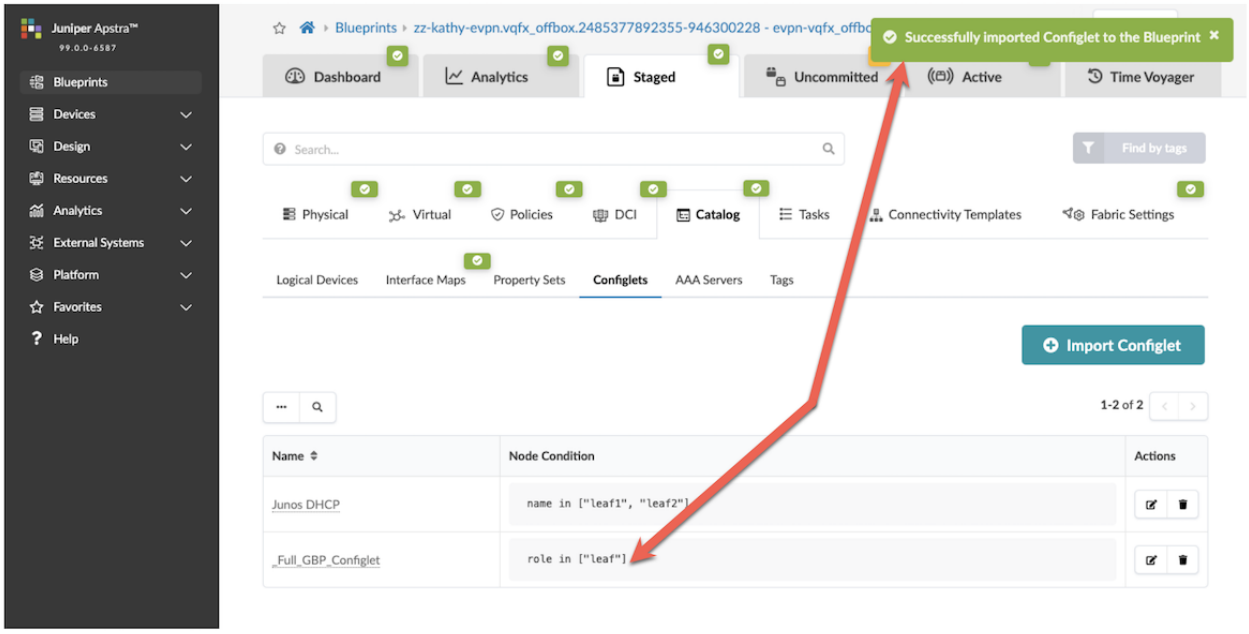


Figure 17: You will now see the configlet imported.

1.7.5 Tagging Ports in Apstra to Enforce GBP

To enable GBP to be enforced on the required interfaces, this configlet requires the generic systems to be tagged. This may seem counter-intuitive however tagging the generic system ensures all interfaces connected to the fabric are enforced the same way.

1.7.6 Using System Node Tags

The final step, to apply the GBP configuration to your given interfaces is to add a node tag.

1. Go to **Staged » Physical » Topology**, select the generic system you wish to add the tag to, then select the node toggle box. Here you can select **Update Node Tags**.

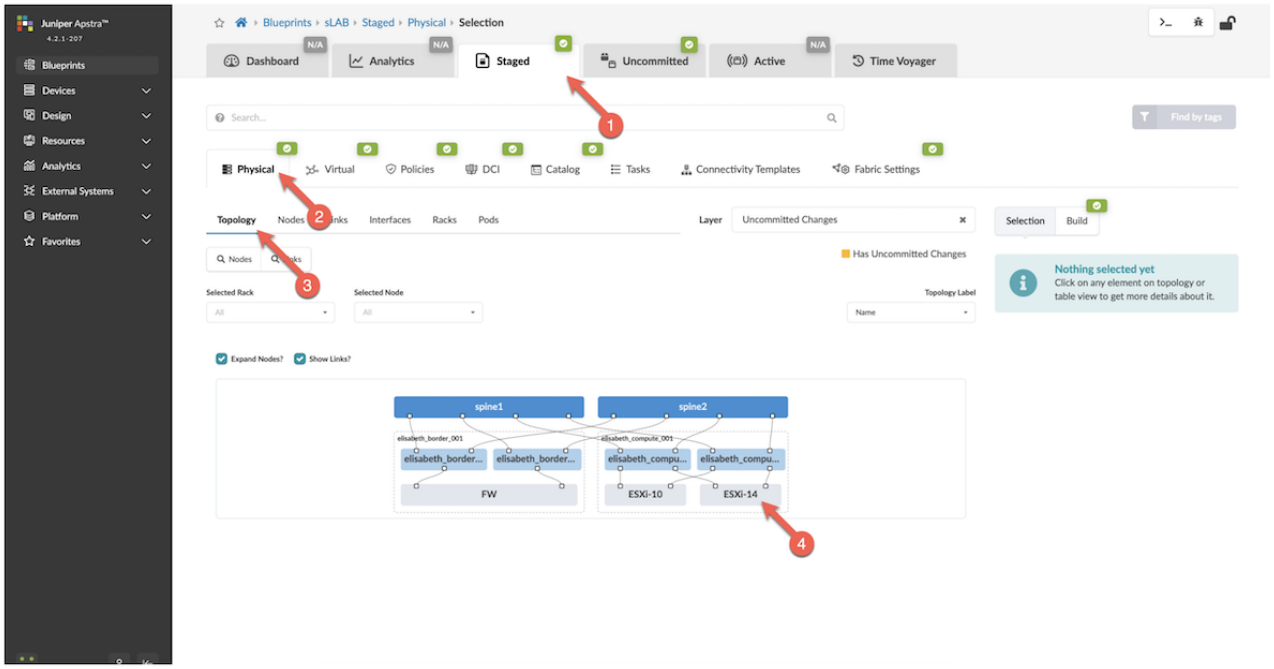


Figure 18: Update Node Tags

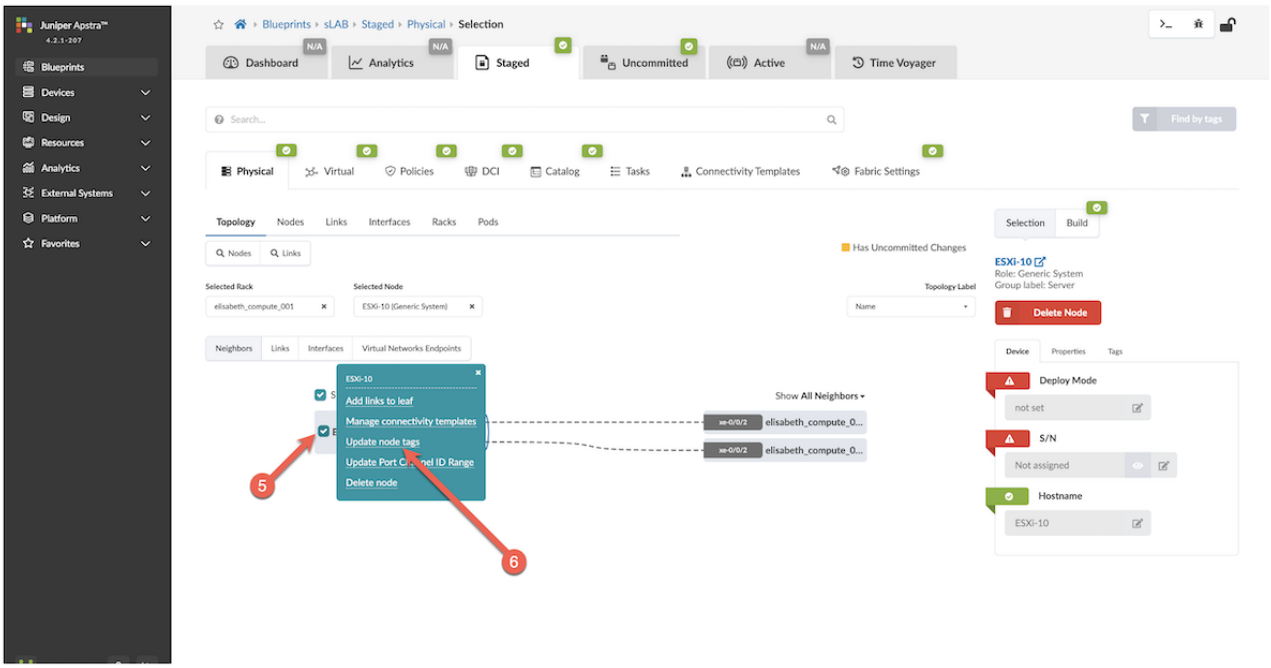


Figure 19: Update Node Tags

2. In this menu you can now add the tags you like. The format for the tags must be “gbp_” followed by the source tag value, for example “gbp_10”. This is the format the configlet is expecting. The source tag must also be present in your property set.



Figure 20: add the tags you like

3. You will now see the tag assigned to the generic system

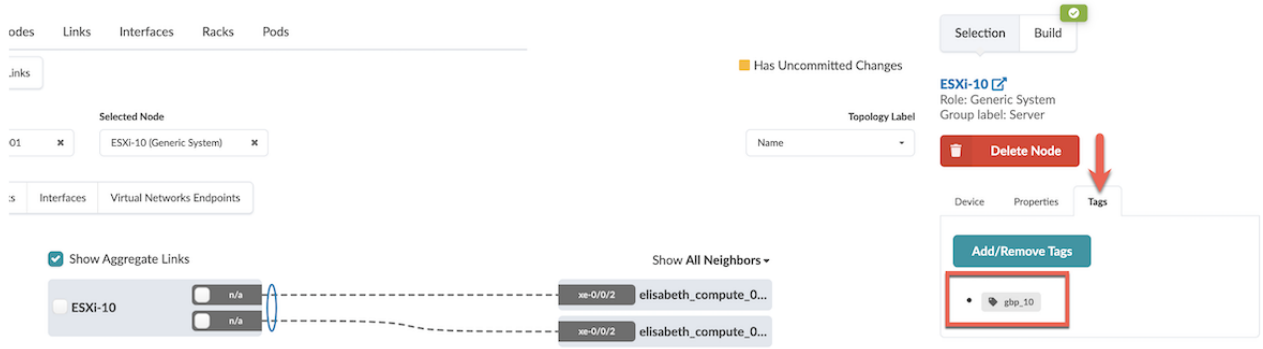
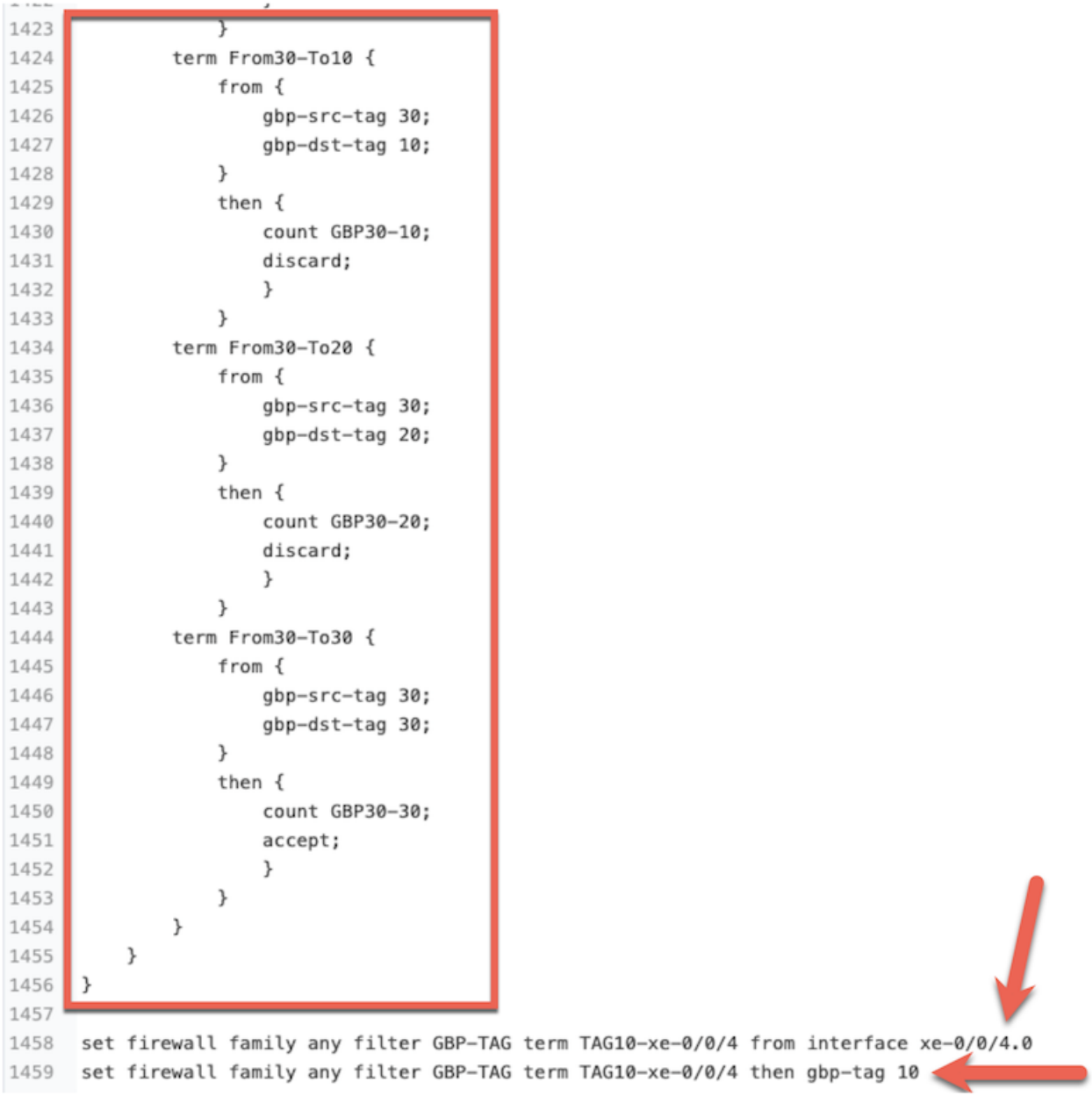


Figure 21: You will now see the tag assigned to the generic system

4. Once applied you can view the switches rendered configuration and at the bottom you will see the new GBP statements for that interface.



```
1423            }
1424        term From30-To10 {
1425            from {
1426                gbp-src-tag 30;
1427                gbp-dst-tag 10;
1428            }
1429            then {
1430                count GBP30-10;
1431                discard;
1432                }
1433            }
1434        term From30-To20 {
1435            from {
1436                gbp-src-tag 30;
1437                gbp-dst-tag 20;
1438            }
1439            then {
1440                count GBP30-20;
1441                discard;
1442                }
1443            }
1444        term From30-To30 {
1445            from {
1446                gbp-src-tag 30;
1447                gbp-dst-tag 30;
1448            }
1449            then {
1450                count GBP30-30;
1451                accept;
1452                }
1453            }
1454        }
1455    }
1456 }
1457
1458 set firewall family any filter GBP-TAG term TAG10-xe-0/0/4 from interface xe-0/0/4.0
1459 set firewall family any filter GBP-TAG term TAG10-xe-0/0/4 then gbp-tag 10
```

Figure 22: you will see the new GBP statements for that interface.

1.8 End Result

The end result of adding both the configlet and property set plus applying the system tag is that the Group Based Policy firewall filters have been created and bound to the relevant interfaces.

1.8.1 Commit-check Procedures

Before committing this configuration to your Juniper device, Apstra allows you to run a commit check. The purpose of this is to ensure that your configlet + Apstra derived configuration is syntactically correct.

The Apstra commit check feature allows administrators to verify configuration changes for errors before applying them to the network. When the commit check command is executed, it validates the proposed

configuration for syntax errors, logical inconsistencies, and hardware compatibility issues.

This feature is useful because it:

1. Prevents Errors: Stops configuration errors from being applied, avoiding potential network issues.
2. Saves Time: Identifies errors early, reducing troubleshooting time later.
3. Ensures Confidence: Provides assurance that changes are correct.
4. Enhances Stability: Contributes to overall network reliability by ensuring error-free configurations.

In essence, the Apstra commit check feature is a tool that helps maintain accurate and reliable network configurations when introducing configlets.

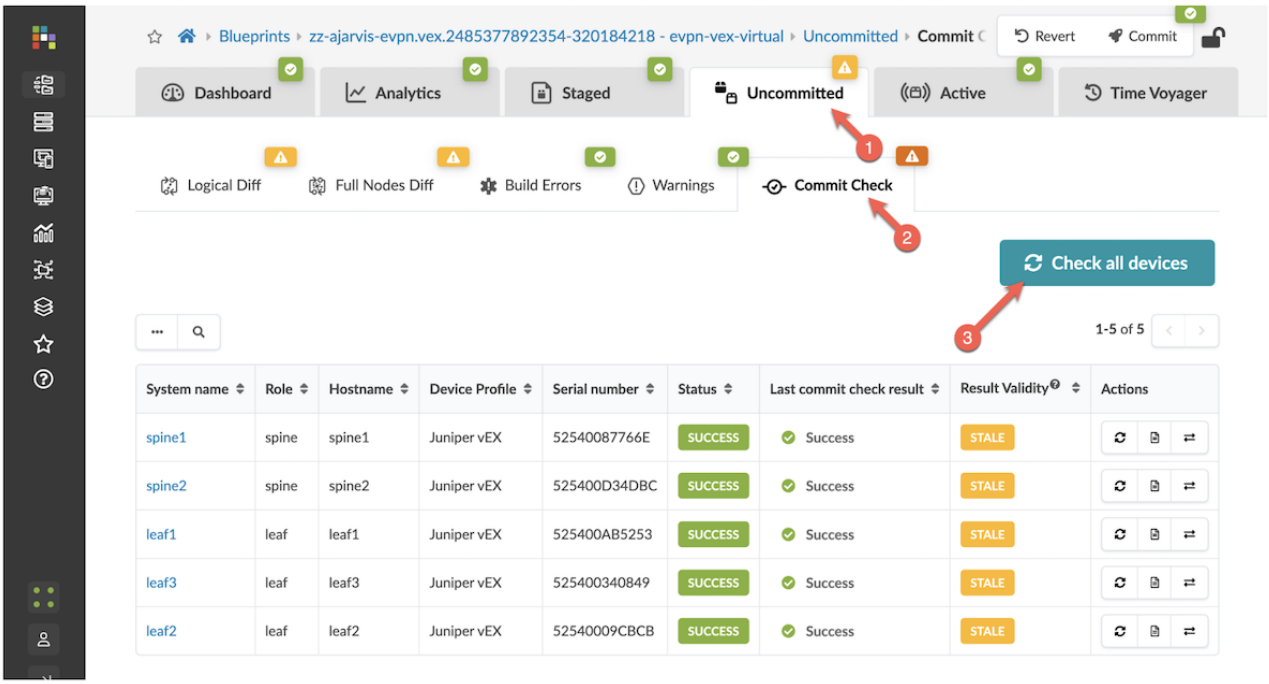


Figure 23: the Apstra commit check feature

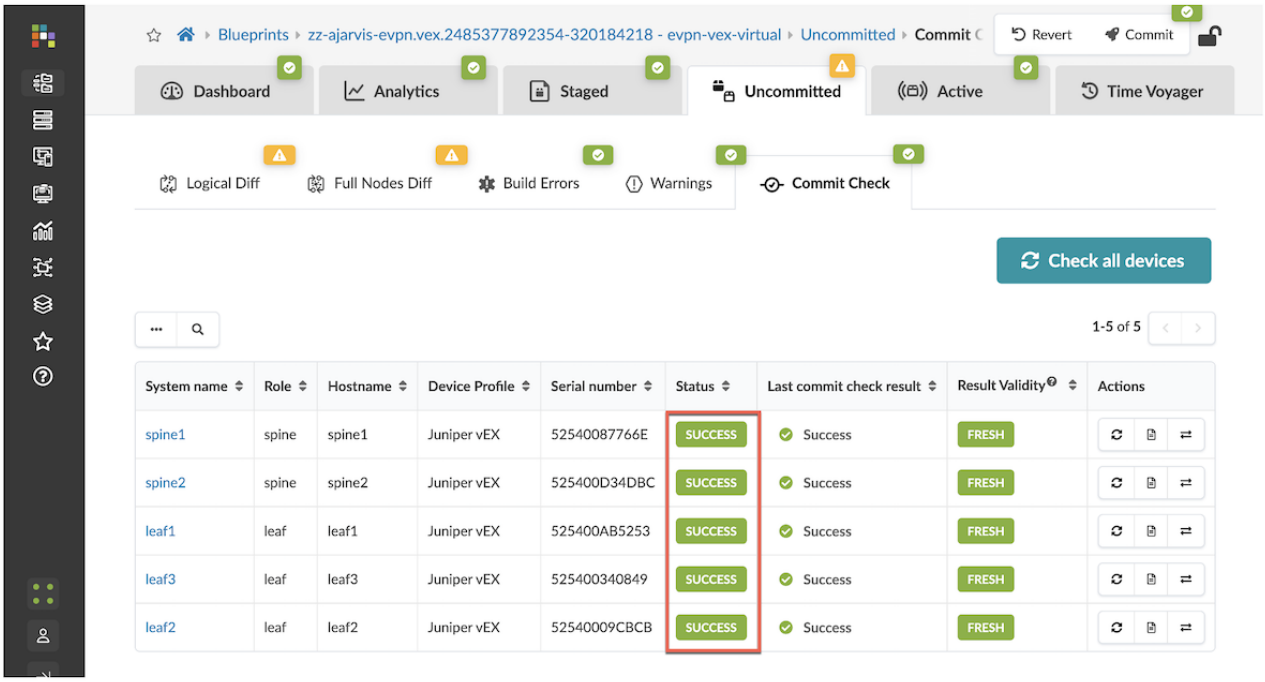


Figure 24: the Apstra commit check feature

1.9 Successful Deployment – End State

After successfully implementing VXLAN GBP in your data center environment using Apstra, the end state should reflect the following key configurations.

1.9.1 Firewall Filter Configuration

The switches in your fabric will have firewall filters applied that govern the communication between different Scalable Group Tags (SGTs). These filters determine which SGTs can communicate with each other and which communications are prohibited.

```
firewall {
   family any {
      filter MSEG {
         term From10-To10 {
            from {
               gbp-src-tag 10;
               gbp-dst-tag 10;
            }
            then {
               count 10-To10;
               accept;
            }
         }
   <<--omitted from output-->>
         term From30-To10 {
            from {
               gbp-src-tag 30;
               gbp-dst-tag 10;
            }
            then {
               count 30-To10;
               discard;
            }
         }
      }
   }
```

```
}
```

1.9.2 SGT to Interface Binding

Each relevant interface or aggregated Ethernet (AE) interface will be bound to a specific SGT. This binding ensures that traffic originating from or destined to these interfaces is appropriately tagged for policy enforcement.

```
firewall {
    family any {
        filter GBP-TAG {
            micro-segmentation;
            term TAG10-ae1 {
                from {
                    interface ae1.0;
                }
                then gbp-tag 10;
            }
            term TAG30-ae2 {
                from {
                    interface ae2.0;
                }
                then gbp-tag 30;
            }
        }
    }
}
```

1.9.3 VXLAN-GBP Profile Activation

The vxlan-gbp-profile will be enabled on all relevant switches in the fabric. This profile activates the necessary hardware resources to support GBP functionality.

```
chassis {
    forwarding-options {
        vxlan-gbp-profile;
    }
}
```

When these configurations are in place and verified, your VXLAN GBP deployment is successfully completed. The network is now capable of enforcing group-based policies, providing micro-segmentation at the fabric level.

Remember that the specific SGTs, policy rules, and interface bindings will depend on your particular network design and security requirements as defined in your Apstra blueprint and property sets.

1.10 Adding Additional GBP Filters in the Future

Adding new GBP filters in the future is a very simple process.

1. Firstly, you update the property set in the global design catalogue to include your GBP configurations.
2. Secondly, within the Blueprint, you re-import the property set.

Once the updated property set is re-imported, all switches that the configlet is applied to will receive the new GBP firewall filter configuration.

1.11 Removing GBP Policies

There may be instances where you need to remove GBP policies in your network. This section outlines the processes for removing SGT to interface bindings and specific GBP firewall filter configurations.

1.11.1 Removing GBP policy from an Interface

To remove an SGT to interface binding, follow these steps:

- Access the Apstra interface and navigate to your blueprint.
- Go to the “Staged” view and select “Physical” > “Topology”.
- Locate the generic system associated with the interface you want to modify.
- Select the node and click on “Update Node Tags”.

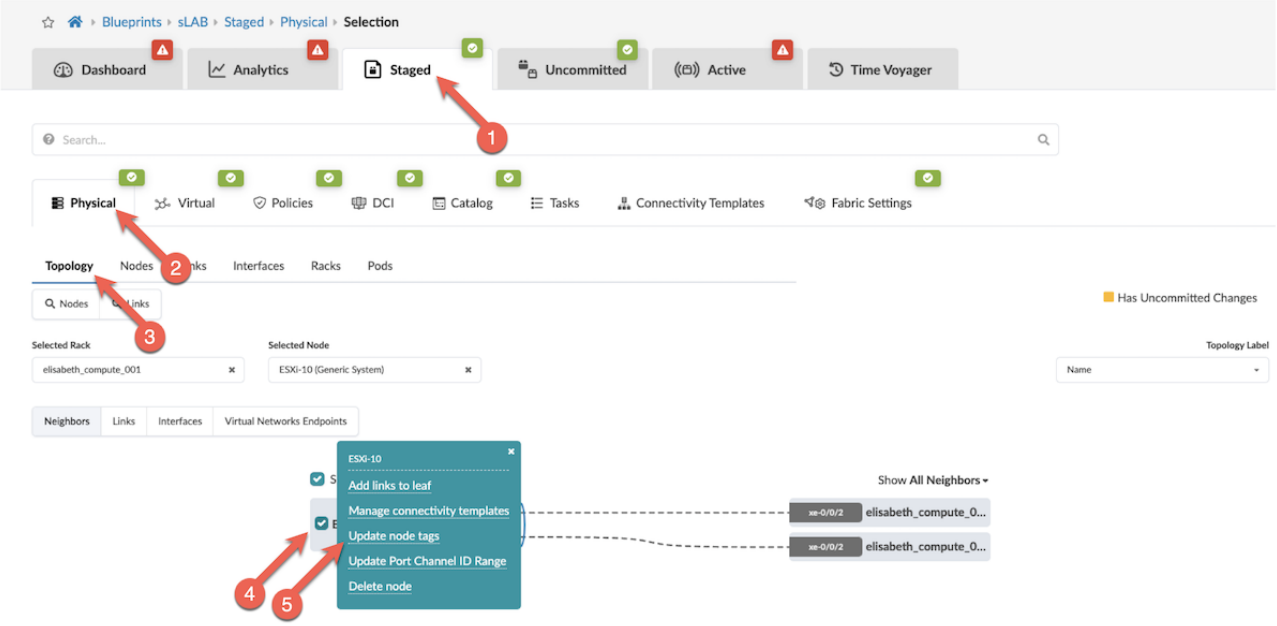


Figure 25: Select the node and click on

- Remove the GBP tag from the interface.
- Commit the changes to your blueprint.

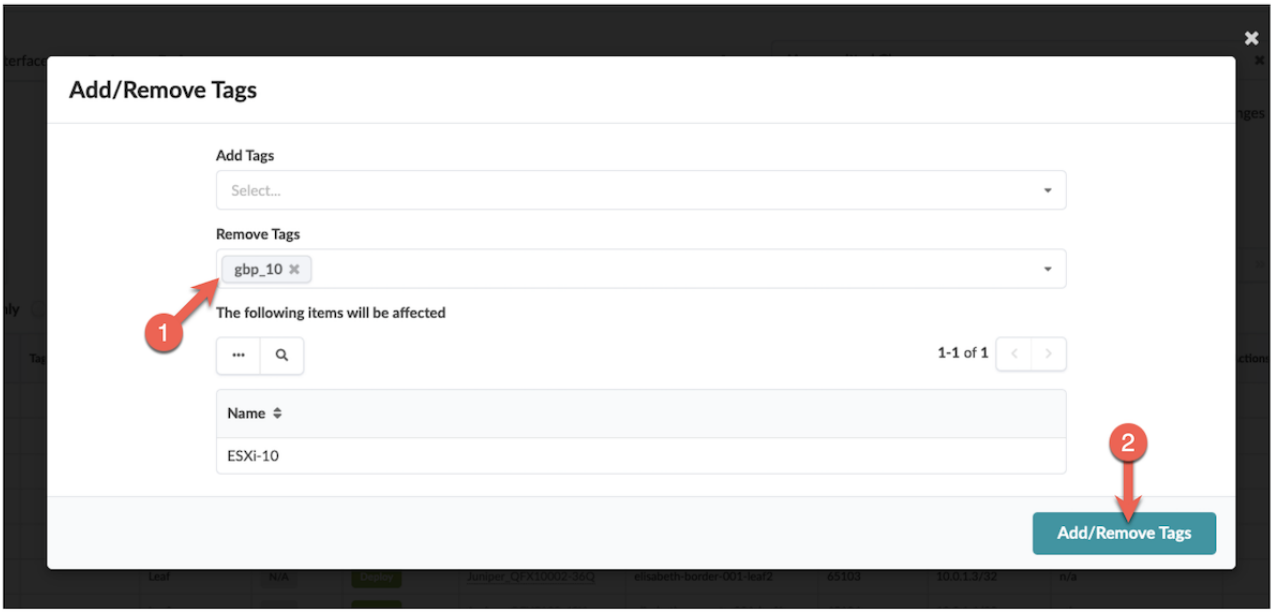


Figure 26: Remove the GBP tag from the interface.

Once these changes are committed and deployed, Apstra will automatically remove the SGT binding from the specified interface in the network devices.

1.11.2 Removing Specific GBP Firewall Filter Configuration

To remove firewall filters you need to:

1. Firstly, you update the property set in the global design catalogue to delete the specific GBP configurations.
2. Secondly, within the Blueprint, you re-import the property set.

After deployment, Apstra will automatically remove the applicable firewall filters from the affected switches in your fabric.

1.12 Validate

This section gives some basic ways to validate that your configuration is working as expected.

This is an output of Juniper's Paragon Active Assurance that was used to validate traffic flows before and after GBP confirmation was pushed.

1. Shows how traffic was dropped when the GBP configuration was pushed to the device
2. Shows other traffic, not denied by GBP, continuing as normal.

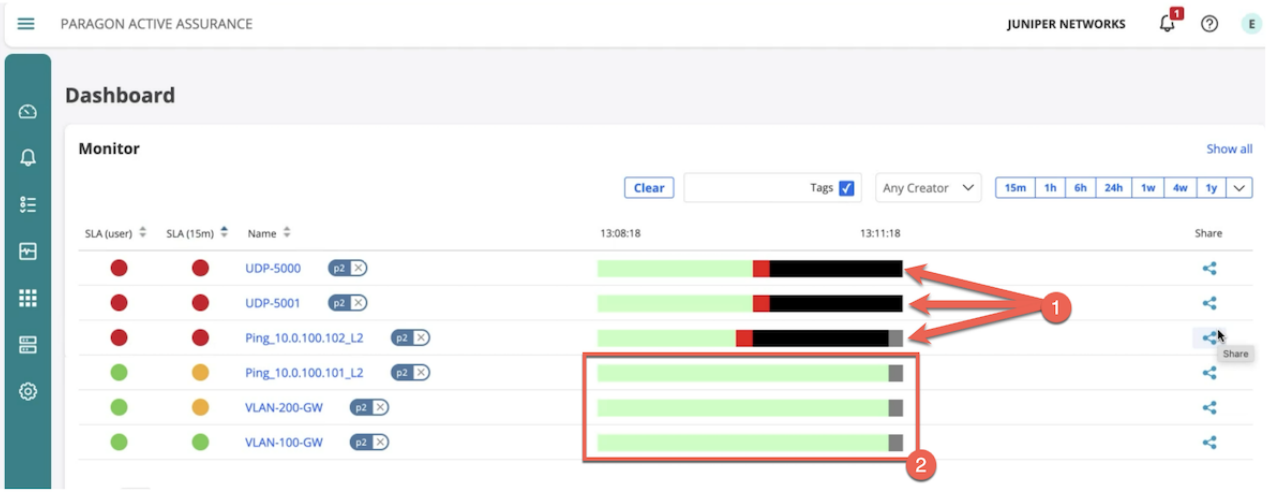


Figure 27: Shows other traffic, not denied by GBP, continuing as normal

This was the expected result of the test.

1.12.1 Command: show ethernet-switching table

This command shows the mapping of GBP tags to specific MAC addresses.

```
{master:0}
lab@leaf1> show ethernet-switching table

MAC flags (S - static MAC, D - dynamic MAC, L - locally learned, P - Persistent static
           SE - statistics enabled, NM - non configured MAC, R - remote PE MAC, O - ovsdb MAC)

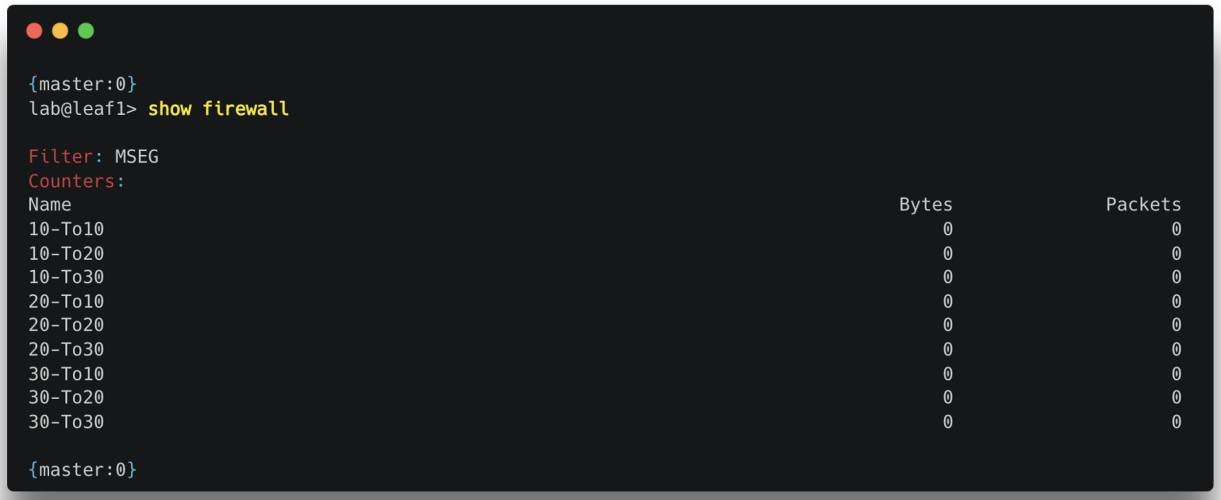

Ethernet switching table : 3 entries, 3 learned
Routing instance : evpn-1
   Vlan                MAC                 MAC       GBP    Logical                 SVLBNH/      Active
   name                address             flags     tag    interface               VENH Index   source
   vn100               00:50:56:87:19:84   DLR         30   ae2.0
   vn100               00:50:56:90:36:2c   DLR         10   ae1.0
   vn200               00:50:56:9:13:05    DLR         30   ae2.0

{master:0}
```

Figure 28: show ethernet-switching table

1.12.2 Command: show firewall

This action is supported only for MAC-based and IP-based GBP policy entries, in our case these counters will not increase due to our GBP application being done on a per-interface basis.



```
{master:0}
lab@leaf1> show firewall

Filter: MSEG
Counters:
Name                                                Bytes              Packets
10-To10                                                 0                    0
10-To20                                                 0                    0
10-To30                                                 0                    0
20-To10                                                 0                    0
20-To20                                                 0                    0
20-To30                                                 0                    0
30-To10                                                 0                    0
30-To20                                                 0                    0
30-To30                                                 0                    0

{master:0}
```

Figure 29: show firewall

1.13 Useful links

- https://apps.juniper.net/feature-explorer/feature-info.html?fKey=11807&fn=VXLAN+group-based+policy+with+ingress+and+egress+configuration
- https://datatracker.ietf.org/doc/html/draft-smith-vxlan-group-policy-00
- https://www.juniper.net/documentation/us/en/software/junos/evpn-vxlan/topics/example/micro-segmentation-using-group-based-policy.html
- https://apps.juniper.net/feature-explorer/feature-info.html?fKey=11807&fn=VXLAN+group-based+policy+with+ingress+and+egress+configuration
- https://www.juniper.net/documentation/us/en/software/junos/evpn-vxlan/topics/example/micro-segmentation-using-group-based-policy.html
- https://techpost.hpe.com/articles/using-apstra-policy-assurance/article
- https://www.juniper.net/documentation/us/en/software/junos/cli-reference/topics/ref/statement/vxlan-gbp-profile.html
- https://github.com/Juniper-SE/Apstra-configlets/tree/main/Juniper/JUNOS/VXLAN_Interface_Based_GBP

1.14 Glossary

- ACL (Access Control List): A set of rules that control network traffic based on predefined criteria.
- Juniper Apstra: A network automation and intent-based networking platform.
- Configlet: In Apstra, a tool for pushing additional custom configurations to network devices.
- Data Center: A facility used to house computer systems and associated components, such as telecommunications and storage systems.
- EVPN (Ethernet VPN): A network technology that provides Ethernet multipoint services over IP/MPLS networks.
- GBP (Group-Based Policy): A network policy framework that manages security and traffic based on groups of endpoints.

- Ingress Enforcement: The application of security policies as traffic enters a network.
- Egress Enforcement: The application of security policies as traffic exits a network.
- Jinja2: A templating engine for Python, often used in network automation.
- Junos: The operating system used in Juniper Networks devices.
- Macro Segmentation: The division of a network into large, broad segments or zones.
- Micro Segmentation: The creation of small, precise segments within a network for fine-grained security control.
- Property Set: In Apstra, a collection of data that defines specific properties, often used with configlets.
- SGT (Scalable Group Tag): A tag used in VXLAN-GBP to classify and enforce policies on network traffic.
- System Tags: In Apstra, user-defined metadata that can be applied to various network elements.
- TCAM (Ternary Content-Addressable Memory): A specialized type of high-speed memory that searches its entire contents in a single clock cycle, used in network devices for rapid lookup of routing information.
- VXLAN (Virtual Extensible LAN): A network virtualization technology that encapsulates Layer 2 Ethernet frames within Layer 4 UDP datagrams.
- YAML: A human-readable data serialization standard often used for configuration files.

1.15 Acknowledgments

This document content has been created and tested by Adam Jarvis and Elisabeth Rodrigues.

Thanks to the reviewers: Kathy Tally, Brian, Adam, Robert Lancaster, Jeremy Wallace, and Jeff Doyle.



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.
Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.
Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com



